

Syrian Private University

Faculty of Artificial Intelligence Engineering

Systems Security and Intelligent Computer

Networks Engineering



Developing a web-based data leak prevention system for local networks

Prepared by

Saud Abdul Aziz Al-Hajji

Supervision by

Dr. Waseem Junaidi

May – 2026

Table of Contents

Table of Contents	2
Table of Figures	6
Table of Tables	6
Chapter One	8
1.1 Introduction	9
1.2 Main Problem	9
1.3 Project Objective	10
1.4 Proposed System	11
1.5 Report Organization	12
1.6 Introduction Conclusion	12
Chapter Two	13
2.1 Introduction	14
2.2 What is the Concept of DLP?	14
2.3 How DLP Works	15
2.3.1 Core Stages of How a DLP System Works	15
2.3.2 Stage One: Discovery	15
2.3.3 Stage Two: Classification	16
2.3.4 Stage Three: Policy Definition	18
2.3.5 Stage Four: Monitoring	19
2.3.6 Stage Five: Analysis	20
2.3.7 Stage Six: Response	21
2.4 Types of DLP	21
2.4.1 Classification Based on Deployment Location	21
2.4.2 Classification Based on Protection Approach	27
2.4.3 Classification Based on Data State	27
2.4.4 Classification Based on Vendor Source	27
2.4.6 Integrated Deployment Model	29
2.4.7 Criteria for Selecting the Appropriate DLP Type	30
2.5 What Is a DLP System?	30
2.6 How DLP Works in Detail According to Deployment Environment	30
2.6.1 On Servers	30
2.6.2 On Browsers	31

2.6.3 On Employee Endpoints	31
2.7 Capturing and Analyzing Internal Traffic	31
2.8 Deep Content Inspection	32
2.9 Session Tracking and Reconstruction	32
2.10 Immediate Alerting and Response Mechanisms	33
2.11 Data Classification	33
2.12 Policy Enforcement	33
2.13 State of the Art	34
2.14 Reference Studies	36
2.15 Summary	37
Chapter Three	39
3.1 Introduction	40
3.2 Laboratory Environment	41
3.3 Network Configuration and Connectivity Verification	43
3.4 File Server Setup on PC 1	46
3.5 Company File Generation	48
3.6 MITMproxy Setup on PC 2	50
3.7 Verifying the Traffic Path	51
3.8 User Profile Preparation	51
3.9 DLP Engine Architecture	53
3.10 Request Processing and Action Detection	55
3.11 Sensitivity AI Component	56
3.11.1 Sensitivity Dataset	56
3.11.2 Feature Extraction Using TF-IDF	57
3.11.3 Trained Models	57
3.11.4 Sensitivity AI Workflow	58
3.11.5 Critical Keyword Safety Override	58
3.12 Behavior AI Component	58
3.12.1 Role-Based Behavior Concept	59
3.12.2 Behavior Score Dataset	59
3.12.3 Encoding and Algorithm	60
3.12.4 Behavior AI Workflow	60
3.13 Risk Scoring Equation	61

3.13.1 Action Risk	61
3.13.2 Decision Thresholds	62
3.14 Enforcement Policy	63
3.14.1 Decision vs Enforcement	63
3.14.2 Enforcement Rules	63
3.14.3 Critical Content Policy	64
3.15 Chapter Summary	64
Chapter Four	66
4.1 Introduction	67
4.2 Testing Methodology	67
4.3 Test Matrix Design	68
4.4 Role-Based Test Scenarios	71
4.5 Action-Based Test Scenarios	73
4.6 Critical Content Test Scenarios	74
4.7 AI Model Evaluation	75
4.7.1 Behavior AI Evaluation	76
4.7.2 Sensitivity AI Evaluation	76
4.7.3 AI Output Role in the Final Decision	77
4.8 DLP Decision Report Analysis	78
4.9 Final Execution Results	79
Chapter Five	82
5.1 Chapter Introduction	83
5.2 Project Summary	83
5.3 Achieved Objectives	84
5.3.1 Internal LAN DLP Gateway	84
5.3.2 Company File Server	85
5.3.3 Content Sensitivity Analysis	85
5.3.4 Role-Based Behavior Analysis	85
5.3.5 Action-Based Risk Evaluation	85
5.3.6 Explainable Decision Engine	86
5.3.7 Structured Logging and Reporting	86
5.4 Main Contributions	87
5.5 Challenges Faced	87

5.5.1 Proxy Routing and Traffic Path Verification	87
5.5.2 File Server Port and Permissions	87
5.5.3 Query Parameter Handling	88
5.5.4 PowerShell URL Construction.....	88
5.5.5 Designing Synthetic Datasets	88
5.5.6 Balancing WARN and BLOCK Decisions.....	88
5.5.7 Critical Content Handling	88
5.6 Limitations	88
5.6.1 Synthetic Datasets	88
5.6.2 HTTP-Focused Implementation	88
5.6.3 Simulated File Actions	89
5.6.4 Small-Scale Laboratory Environment	89
5.6.5 No Production GUI Dashboard.....	89
5.6.6 Limited User Identity Integration	89
5.6.7 No Real-Time Administrator Alerting	89
5.7 Future Work.....	89
5.7.1 Support Real File Operations	89
5.7.2 Add SMB and FTP Support.....	89
5.7.3 Integrate with Active Directory	90
5.7.4 Build a Graphical Dashboard	90
5.7.5 Improve Sensitivity AI with Real Data	90
5.7.6 Improve Behavior AI with Real User Logs	90
5.7.7 Add SIEM Integration	90
5.7.8 Add Real-Time Alerting	90
5.7.9 Add Policy Management Interface	91
5.7.10 Performance and Scalability Testing	91
5.8 Final Conclusion	91
References	92
References for chapter 1.....	92
References for chapter 2.....	92
References for chapter 3.....	93

Table of Figures

Figure 1 : General Data Leakage Prevention System	14
Figure 2 : Types of classification in DLP	17
Figure 3 : Types of DLP according to publishing site	22
Figure 4 : Endpoint DLP at the endpoint level	23
Figure 5 : CASB Working Mechanism	26
Figure 6 : General structure of the DLP system	29
Figure 7 : Top-tier DLP brands on the market, based on pricing.....	34
Figure 8 : Lab- Topology	41
Figure 9 : Server Static IP Assigning.....	42
Figure 10 : D.L.P Static IP Assigning.....	42
Figure 11 : Normal User PC IP	43
Figure 12 : Pinging 1	44
Figure 13 : Pinging 2	44
Figure 14 : Pinging 3	45
Figure 15 : Successful CURL Command	46
Figure 16 : Server Initialization	47
Figure 17 : Example request	47
Figure 18 : Expected Server Output	48
Figure 19 : user profiler selector	52
Figure 20 : Active profile	53
Figure 21 : processing flow	54
Figure 22 : Final risk in real time	62
Figure 23 : Basic Used Command - Read	68
Figure 24 : Basic Used Command - Delete	68
Figure 25 : Setting up the role for testing	69
Figure 26 : Manual Test	70
Figure 27 : Active Profile.JSON.....	71
Figure 28 : DLP Listening to a user with role Sales	72
Figure 29 : Action Rejection / Blocking based on risk score.....	77
Figure 30 : DLP Warning	81

Table of Tables

Table 1 : Response mechanisms	15
Table 2 : Classification of detection mechanisms	16
Table 3 : Typical classification levels	18
Table 4 : Response mechanisms	21
Table 5 : Network-level advantages of DLP.....	22
Table 6 : Challenges facing DLP at the network level	23
Table 7 : Endpoint monitoring components	24
Table 8 : Advantages of Endpoint Monitoring Components	24
Table 9 : Challenges facing monitoring components on Endpoint.....	25

Table 10 : Types of Cloud DLP	25
Table 11 : CASB Advantages	26
Table 12 : CASB Challenges	26
Table 13 : Types of protection methods	27
Table 14 : Types of data that DLP handles	27
Table 15 : Classifying DLP solutions by source nature	28
Table 16 : General comparison between DLP types.....	28
Table 17 : Factors influencing DLP selection	30
Table 18 : State of the Art	35
Table 19 : Reference studies	37
Table 20 : Laboratory Devices.....	42
Table 21 : Laboratory Characteristics.....	43
Table 22 : Static IP Addressing Scheme.....	44
Table 23 : File Server Configuration	48
Table 24 : Company Files	49
Table 25 : Purpose of Files	50
Table 26 : MITMproxy Configuration	50
Table 27 : Proxy Verification Results	51
Table 28 : action table.....	55
Table 29 : dataset features	57
Table 30 : dataset features -2 example	57
Table 31 : models.....	57
Table 32 : Role Based Examples	59
Table 33 : Behavior Dataset	60
Table 34 : behavior dataset sample	60
Table 35 : risk components.....	61
Table 36 : Action risk score	62
Table 37 : Decision Score	62
Table 38 : Enforcement Rules	64
Table 39 : content policy.....	64
Table 40 : Testing Components	68
Table 41 : Representative Test Files	70
Table 42 : Test Matrix Dimensions	71
Table 43 : Role-Based Scenario Examples	72
Table 44 : Expected Behavior Risk by Role Context	73

Chapter One

-Introduction-

1.1 Introduction

In the era of rapid digital transformation, data has become the most valuable strategic resource for organizations, forming the backbone of their operations and competitive decisions. With this increasing reliance on digital infrastructures, the security risks associated with the integrity and confidentiality of this data are escalating, particularly within Local Area Network (LAN) environments, which represent the beating heart of any organization. In this context, Data Leakage Prevention (DLP) systems emerge as a fundamental pillar of modern cybersecurity strategies, representing an advanced line of defense aimed at ensuring that sensitive information does not leave the internal network boundaries, whether intentionally by a malicious user or unintentionally due to human error.

Traditional protection based on firewalls and Intrusion Detection Systems (IDS) is no longer sufficient to face contemporary challenges, as these systems often focus on threats coming from outside the network, while overlooking the risks of internal data leakage that may occur through communication channels between devices within the same local network. This security vacuum creates a dangerous gap, where sensitive information can be transported via legitimate protocols such as SMB, FTP, HTTP, or SMB without being captured by traditional monitoring tools, leaving organizations vulnerable to the loss of their intellectual and financial assets without their knowledge.

This project comes to bridge this gap by designing and implementing a specialized Data Loss Prevention system for LANs, relying on Deep Packet Inspection (DPI) techniques and real-time session tracking. The proposed system provides a unique combination of internal traffic capture capabilities and dynamic content analysis, with a flexible security policy management framework, enabling organizations to enforce strict control over the flow of their sensitive data within their internal perimeter. By providing a comprehensive and transparent view of data behavior (Situational Awareness), this project contributes to enhancing situational awareness for network administrators and represents a qualitative addition to cybersecurity tools dedicated to internal environments.

1.2 Main Problem

Despite the significant development witnessed by information security solutions in recent years, most research and commercial efforts have focused on protecting the network perimeter from external breaches, leaving a clear security gap related to monitoring data movement within the Local Area Network (LAN). The core problem lies in the fact that sensitive data can be transferred between internal devices using ostensibly legitimate protocols and services, making the detection of leakage attempts extremely difficult using traditional tools.

This problem manifests itself under several factors:

- **Diversity of internal transport channels:** Organizations rely on multiple protocols for file transfer and data sharing within the network, such as SMB,

FTP, HTTP, and SMB, creating a wide attack surface that is difficult to cover with traditional monitoring tools.

- **Inadequacy of current analysis tools:** Despite the power of tools like Wireshark and Tcpdump, they require manual intervention and high technical expertise to analyze traffic and detect leakage incidents. They also lack automated mechanisms for enforcing security policies or providing an immediate response to violations.
- **Absence of content inspection mechanism:** Most monitoring solutions rely on analyzing only packet headers, without delving into the payload content, rendering them unable to detect sensitive data hidden within transferred files or communication texts.
- **Lack of immediate response mechanisms:** Current solutions often lack the ability to take proactive actions upon detecting a leakage attempt, such as terminating the violating session (Session Termination) or issuing an immediate alert to contain the incident before data transfer is complete.

Based on this, the research problem crystallizes in the urgent need for a specialized LAN DLP system capable of deep content analysis, effective session tracking, and flexible security policy enforcement, while providing alerting and immediate response mechanisms that enable organizations to maintain complete control over their sensitive data internally.

1.3 Project Objective

This research aims to achieve a primary goal: designing and implementing an advanced, specialized Data Loss Prevention (DLP) system for Local Area Network (LAN) environments, characterized by efficiency, flexibility, and the ability to integrate with existing infrastructures. A set of detailed objectives branches from this main goal:

- **Internal Traffic Capture and Analysis:** Develop a module capable of capturing data packets from local network interfaces in real-time, supporting advanced filtering based on source/destination addresses, ports, and protocols.
- **Deep Packet Inspection (DPI):** Implement a specialized mechanism for inspecting packet payloads, searching for predefined sensitive data patterns using Regular Expressions and keyword lists, including the identification of credit card numbers, leaked documents, and sensitive terminology.
- **Session Tracking & Reassembly:** Build an engine capable of assembling fragmented packets (TCP/UDP), reassembling high-level session content (e.g., file transfers via HTTP/FTP), and providing a complete context for the data transfer process.
- **Flexible Policy Management Framework:** Create an interface allowing the system administrator to define dynamic security policies that specify sensitive data sources, allowed or denied destinations, protocols subject to monitoring, and the action to be taken upon policy violation (alert, log, block).

- **Real-time Alerting & Response:** Design an instant alerting system that notifies administrators upon detecting any DLP policy violation, with the ability to execute automatic proactive actions such as terminating the violating session by sending RST packets.
- **Data Visualization & Reporting:** Create a Graphical User Interface (GUI) displaying a central dashboard with live traffic statistics, alert logs, active sessions, and comprehensive analytical reports that support security auditing processes.

1.4 Proposed System

The proposed system in this research is a specialized tool for Data Loss Prevention in Local Area Networks (LAN DLP System), developed using the Python programming language due to its rich libraries for network handling and data analysis. The system is based on a modular architecture consisting of six main integrated modules:

- **Packet Capture Module:** Built on the Scapy library, operating in promiscuous mode to capture all traffic on the local network. This module supports pre-filtering using Berkeley Packet Filter (BPF) to reduce load on the system.
- **Content Inspection Engine:** Performs protocol decapsulation and packet payload inspection using efficient search algorithms. The engine includes an embedded, expandable database of sensitive data patterns (Patterns) that can be updated dynamically.
- **Session Tracking & Reassembly Engine:** This engine operates at a higher level than individual data packets, linking packets related to a single communication session and rebuilding the complete session content (e.g., a transferred file or web page) for the Content Inspection Engine to analyze as a whole.
- **Policy Management Engine:** A central database for security rules, where policies are stored in a structured format (JSON/SQLite). Policies support multiple conditions including (source, destination, protocol, sensitive data type, and required action).
- **Alerting & Response Module:** Responsible for generating immediate notifications upon detecting a violation. Response mechanisms include:
 - **Logging:** Saving incident details in an alerts database.
 - **Notification:** Displaying a pop-up alert in the graphical user interface.
 - **Active Response:** Sending RST packets to terminate violating TCP sessions, or executing custom commands (e.g., updating firewall rules).
- **Graphical User Interface (GUI):** A comprehensive control panel developed using Tkinter and TTKBootstrap libraries, providing the administrator with a centralized view of the system status. The interface includes:
 - A main dashboard displaying live statistics.
 - Policy Editor window.
 - Searchable and filterable Alert Log.
 - Viewer of active sessions and historical reports.

1.5 Report Organization

This report is organized according to the academic structure adopted in research studies, consisting of seven main chapters:

- **Chapter One: Introduction:** This chapter presents the general framework of the research, starting with the academic and applied context of the data loss prevention problem in local networks, moving through the identification of the research problem and objectives, to providing an overview of the proposed system and the report structure.
- **Chapter Two: Basic Concepts and Reference Studies:** This chapter reviews the theoretical foundations upon which the research is built, including the analysis of DLP systems, deep packet inspection techniques, and relevant network protocols. It also provides a critical analysis of the State of the Art in this field, compares existing commercial and academic solutions, and identifies the research gap that this project addresses.
- **Chapter Three: Analytical Study:** Includes an in-depth analysis of functional and non-functional system requirements, in addition to studying threat modeling specific to data leakage in internal networks, and analyzing main use cases.
- **Chapter Four: Design Study:** Covers the proposed system's architecture, data flow diagrams between different modules, database design details, the structure of the graphical user interface, and the logic of the policy engine and content inspection mechanism.
- **Chapter Five: Implementation and Testing:** Describes the practical aspects of system development, including the development environment used, algorithms implemented in each module, and programming details of core system components such as packet capture, session tracking, content inspection, and active blocking mechanism.

1.6 Introduction Conclusion

This chapter provided the general framework for the project of designing and developing a specialized Data Loss Prevention (DLP) system for local networks (LAN). It reviewed the importance of this type of system amid the increasing reliance on data as a strategic asset and the exacerbation of internal leakage risks that traditional security solutions do not adequately cover. The research problem was precisely identified, represented by the absence of effective tools focusing on internal traffic that combine deep content inspection, session tracking, and immediate response mechanisms. The research objectives were formulated, aiming to bridge this gap through an integrated system based on a flexible modular architecture. A brief description of the proposed system and its core components was provided, including modules for packet capture, content inspection, session tracking, policy management, and alerting/response. Finally, the report structure outlining the organization of the scientific content across its seven chapters was presented. This chapter constitutes the foundation upon which the scientific and technical details in subsequent chapters are built.

Chapter Two

-Basic Concepts and Reference Study-

2.1 Introduction

In the era of rapid digital transformation, data has become the main nerve center of any organization, whether governmental or private. With the increasing reliance on cloud systems, remote work, and the diversity of communication devices, significant challenges have emerged in protecting sensitive data from leakage or unauthorized access. Data Leakage Prevention (DLP) systems come as an integrated strategic solution that does not only involve setting up defensive walls but extends to understanding how data flows inside and outside the organization, classifying it, monitoring it, and taking immediate actions to protect it. This chapter aims to establish the theoretical and cognitive foundations upon which the requirements of the proposed system are built, by reviewing the core concepts of DLP, its working mechanisms, its types, leading to an analysis of the state-of-the-art in this field.

2.2 What is the Concept of DLP?

DLP is an abbreviation for Data Leakage Prevention. It is a set of tools and strategies designed to ensure that authorized users do not send or share sensitive information outside the permitted network boundary, whether accidentally or deliberately. The concept of DLP is not limited to preventing security breaches but also includes detecting and preventing attempts to transfer data that violate organizational policies. DLP acts as an intelligent "surveillance camera" for data movement, analyzing the content, context, and behavior associated with the data to make the appropriate decision (allow, block, or alert).

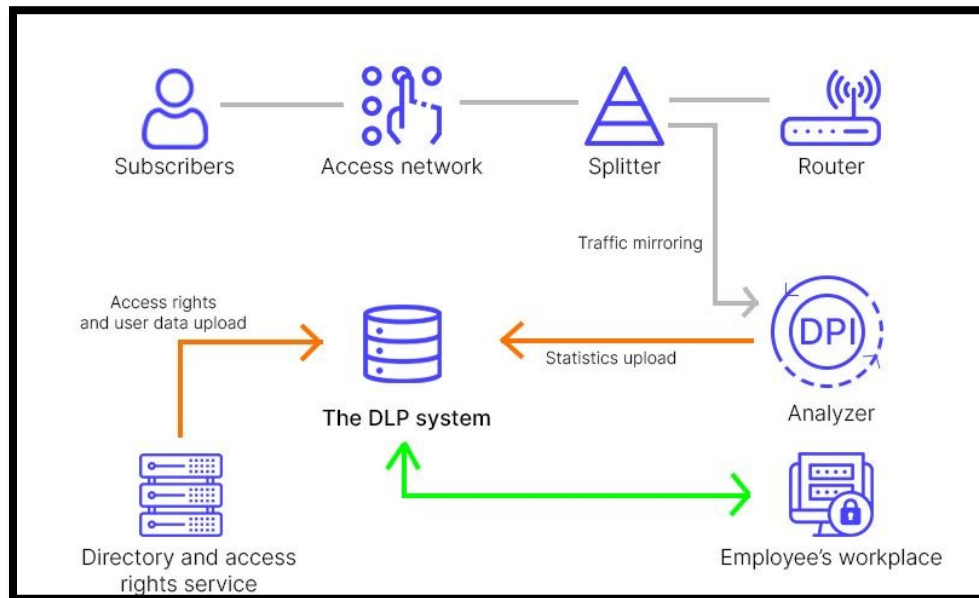


Figure 1 : General Data Leakage Prevention System

2.3 How DLP Works

Data Leakage Prevention systems operate as an integrated framework that passes through several sequential stages, starting from the moment data is created and continuing until an attempt is made to transfer or share it.

To fully understand how DLP works, the process must be divided into systematic stages. Each stage depends on the output of the previous one and integrates with the remaining components of the system. This layered workflow allows DLP solutions to identify sensitive data, classify it, monitor its movement, analyze potential violations, and respond according to predefined security policies.

2.3.1 Core Stages of How a DLP System Works

The operation of a DLP system can be summarized into six main stages that form the security-oriented data processing lifecycle:

Stage	Description	Output
Discovery	Identifying the locations of sensitive data within the organization's environment, such as servers, endpoints, and cloud platforms.	A data map showing where sensitive data resides.
Classification	Analyzing and categorizing data according to its sensitivity level, such as public, internal, confidential, or restricted.	Labeled data with assigned sensitivity categories.
Policy Definition	Defining rules that determine how each category of data should be handled.	An organized and published policy set.
Monitoring	Monitoring data movement across different points, including the network, endpoints, and cloud environments.	Event logs related to data activity.
Analysis	Analyzing data movement against the defined policies.	Decisions such as matches, violations, or false positives.
Response	Applying the appropriate action based on the analysis result.	Blocking, alerting, quarantining, or allowing with logging.

Table 1 : Response mechanisms

2.3.2 Stage One: Discovery

Before a DLP system can protect data, it must first know where that data is located. The discovery stage is a comprehensive scanning process that covers all data repositories within the organization.

Types of Discovery

1. Data-at-Rest Discovery

This includes scanning stored data in several locations:

- **File servers:**
Scanning the content of network shares, such as Windows File Servers and NAS systems.
- **Content management systems:**
Scanning repositories such as SharePoint, Confluence, and Microsoft Teams.
- **Databases:**
Scanning SQL, Oracle, and MySQL tables to identify columns that contain sensitive data, such as national ID numbers or credit card information.
- **User devices:**
Scanning files stored locally on laptops and employee workstations.

2. Data-in-Motion Discovery

This occurs in parallel with continuous traffic monitoring, where sensitive data is detected at the moment an attempt is made to transfer it across the network.

3. Data-in-Cloud Discovery

This involves scanning SaaS applications connected to the organization, including Shadow IT Discovery, in order to identify unauthorized applications that may contain sensitive data.

Discovery Mechanisms

Mechanism	Description	Example
Scheduled Scanning	Periodic scanning of all repositories, such as daily or weekly scans.	Scanning all SharePoint files every Sunday night.
Continuous Monitoring	Real-time monitoring of newly created or modified files.	Immediately scanning any file uploaded to Teams.
Initial Baseline Scan	A one-time comprehensive scan used to create a baseline inventory.	The initial scan performed during system installation.

Table 2 : Classification of detection mechanisms

2.3.3 Stage Two: Classification

After discovering the locations of data, the system analyzes the actual content of that data to determine its sensitivity level. This stage is among the most critical stages, because its accuracy directly determines the effectiveness of the entire system.

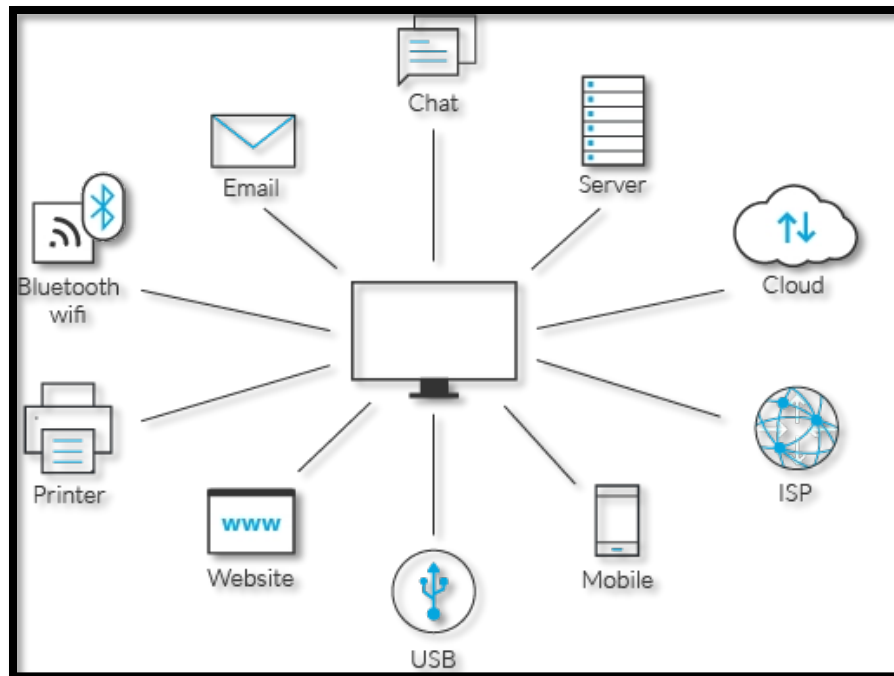


Figure 2 : Types of classification in DLP

Classification Methods

1. Automated Classification

Automated classification can be performed using several techniques:

- **Regular expressions:**
Searching for specific patterns, such as using a numeric pattern to detect national ID numbers.
- **Digital fingerprinting:**
Creating a unique sample or fingerprint from a reference document, such as a confidential contract, and comparing other files against it.
- **Machine learning:**
Training a model to understand the context of data. For example, the model can distinguish between “breast cancer” as a general medical term in an article and “breast cancer patient report” as sensitive medical information.
- **Keywords:**
Searching for predefined words such as “Confidential,” “Highly Confidential,” or similar sensitivity indicators.

2. Manual Classification

In this approach, the user manually selects the sensitivity level of a file when creating or modifying it. A dialog box may appear asking the user: “What is the sensitivity level of this document?” The user may also be reminded of the applicable policies before saving the file.

3. Hybrid Classification

Hybrid classification combines automated and manual classification. The system suggests a classification level, while the user may modify it. Any change made by the user is recorded in the system logs.

Typical Classification Levels:

Level	Description	Examples	Applied Policies
Public	Information that may be publicly shared.	Marketing materials, press releases.	No restrictions.
Internal	Information intended for internal use only.	Internal newsletters, shift schedules.	Prevent sharing outside the internal domain.
Confidential	Sensitive organizational data.	Financial data, supplier contracts.	Block transfer via personal email and encrypt external transmission.
Restricted	Highly sensitive information.	Intellectual property, customer data, HR files.	Complete blocking of transfer outside the permitted scope and immediate security alerting.

Table 3 : Typical classification levels

2.3.4 Stage Three: Policy Definition

A DLP policy is a set of rules that defines **who** can do **what** with **which data**, to **where**, and using **which channel or tool**.

Core Components of a Policy

1. Conditions

Conditions define the scope of the policy:

- **User:**
A specific user, group, job role, or all users.
- **Data:**
The data category, such as credit card numbers, national IDs, or engineering documents; the classification level, such as confidential or internal; or a specific repository.
- **Destination:**
An internal domain, external domain, specific application such as Gmail or Dropbox, IP address, or geographic region.
- **Channel:**
Email, HTTP/HTTPS, FTP, USB, printers, or messaging applications.

2. Actions

The policy may trigger one or more of the following actions:

- **Monitor only:**
Record the event without intervention.

- **Alert:**
Send a notification to the security administrator.
- **Block:**
Prevent the operation completely and display a message to the user.
- **Quarantine:**
Isolate the email or file for manual review.
- **Encrypt:**
Automatically encrypt the email or file before transmission.
- **Educate and notify:**
Display an awareness message to the user and allow the operation after confirmation.

3. Exceptions

Exceptions define specific cases in which a policy violation is explicitly allowed. For example, sending confidential files to partner@trustedcompany.com may be allowed even though external transmission is generally blocked.

2.3.5 Stage Four: Monitoring

The monitoring stage represents the eyes and ears of the system. It tracks data movement across different points within the infrastructure.

Types of Monitoring Points

1. Network Monitoring

A DLP device or virtual appliance is deployed at the network gateway. Traffic can be intercepted using one of the following methods:

- **Transparent bridge mode:**
The device is placed between the router and the switch without requiring network configuration changes.
- **Proxy mode:**
Traffic is routed through a DLP proxy.
- **Port mirroring / SPAN:**
Traffic is copied to a dedicated analysis port without affecting the original data flow.

2. Endpoint Monitoring

A software agent is installed on the user's device. The agent monitors:

- File system operations such as creating, modifying, copying, and moving files.
- Applications that access sensitive files.
- Peripheral devices such as USB, printers, Bluetooth, and file transfer interfaces.
- Clipboard operations such as copying and pasting.

3. Cloud Monitoring

Cloud monitoring is achieved through API integration with SaaS applications such as Microsoft 365, Google Workspace, Slack, and Salesforce.

The system monitors:

- External file sharing.
- Large-volume data downloads.
- Logins from unusual locations.

2.3.6 Stage Five: Analysis

This stage represents the “brain” of the system. Decisions are made based on the data collected from monitoring points and evaluated against the defined policies.

Analysis Techniques

1. Content Analysis

- **Exact matching:**
Comparing a file against a database of reference documents.
- **Partial matching:**
Searching for fragments of sensitive data inside a larger file.
- **Vector analysis:**
Comparing document similarity using artificial intelligence techniques.

2. Contextual Analysis

This technique considers the context of the data activity, such as:

- Who is the user?
Is the user legitimate, compromised, or currently on leave?
- Where is the data being sent?
Is the destination trusted, a competitor, or a high-risk country?
- Which tool is being used?
Is it an approved application or an unauthorized chat tool?
- When is the action occurring?
Is it during working hours, at midnight, or immediately before the user’s resignation?

3. Behavioral Analysis — UEBA

User and Entity Behavior Analytics builds a behavioral baseline for each user and detects anomalies such as:

- **Unusual volume:**
A user who normally sends 10 MB suddenly sends 2 GB.
- **Unusual frequency:**
A user repeatedly downloads data from a database within a short period.
- **Unusual destination:**
A user sends data to a country or destination they do not usually interact with.

4. False Positive Management

False positives are among the greatest challenges in DLP systems. They can be reduced using techniques such as:

- **Whitelists:**
Excluding specific applications or destinations.
- **Continuous learning:**
Updating analysis models based on administrator reviews.
- **Monitor mode:**
Running the system in monitoring mode before enabling blocking, allowing policies to be adjusted and false positives reduced.

2.3.7 Stage Six: Response

When the system determines that an event is a policy violation, or a potential violation, the appropriate response mechanism is activated.

Response Type	Description	When It Is Used
Auto-Block	Immediately prevents the operation without human intervention.	High-risk policies, such as transferring highly confidential data.
Alert Only	Records the event and sends an alert to the administrator.	Medium-risk policies or early monitoring stages.
Quarantine	Isolates the email or file in a holding area for administrator review.	Cases where the decision is unclear and requires human review.
Allow with Log	Allows the operation but records it for later auditing.	Low-risk policies or authorized users.
Interactive Education	Displays a message explaining the issue and how to correct it, then allows the operation after confirmation.	Used to improve security awareness and reduce disruptive blocking.

Table 4 : Response mechanisms

2.4 Types of DLP

DLP systems can be classified according to several criteria, mainly deployment location, protection approach, and the state of the targeted data. Building an integrated DLP system requires a deep understanding of these classifications in order to select the appropriate combination for the organization's needs.

2.4.1 Classification Based on Deployment Location

This is the most common classification in the industry. It divides DLP solutions into three main types based on where monitoring and enforcement components are deployed.

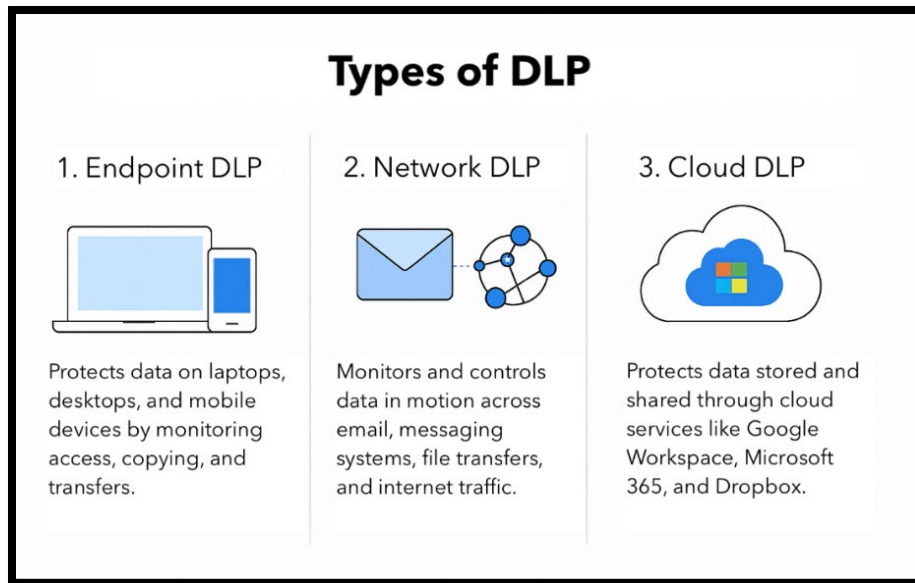


Figure 3 : Types of DLP according to publishing site

2.4.1.1 Network DLP

Definition

Network DLP is deployed as a physical appliance or virtual appliance at network egress points to monitor and analyze incoming and outgoing traffic. It functions as a security gateway that inspects every packet entering or leaving the organization.

Working Mechanism

A Network DLP device is placed inline between the main router and the internal network. It inspects multiple protocols, including SMTP for email, HTTP/HTTPS for web traffic, FTP for file transfer, and file-sharing protocols such as SMB.

It uses techniques such as Deep Packet Inspection and SSL/TLS interception to inspect encrypted content where applicable.

Advantages

Advantage	Description
Comprehensive coverage	Covers all devices connected to the network without installing software on each device.
Centralized control	Policies are managed and updated from a single point.
Operating-system independent	Works with Windows, macOS, Linux, mobile devices, and unmanaged devices.
Cloud application monitoring	Detects Shadow IT applications and prevents data transfer to them.

Table 5 : Network-level advantages of DLP

Challenges

Challenge	Description
Increasing encryption	With TLS 1.3 and end-to-end encryption, content inspection becomes more difficult.
Latency	It may introduce delay if not properly designed for traffic volume.
Limited protection for disconnected devices	It does not protect devices outside the network, such as remote workers.
Limited peripheral control	It cannot control USB ports or local printers.

Table 6 : Challenges facing DLP at the network level

Suitable Use Cases

Network DLP is suitable for:

- Organizations that rely heavily on email and cloud applications.
- Environments containing unmanaged BYOD devices.
- Monitoring outbound traffic from data centers and servers.

2.4.1.2 Endpoint DLP

Definition

Endpoint DLP is installed as a software agent on end-user devices, such as laptops, desktops, servers, and mobile devices. It works independently of network connectivity, making it highly suitable for remote work and distributed environments.

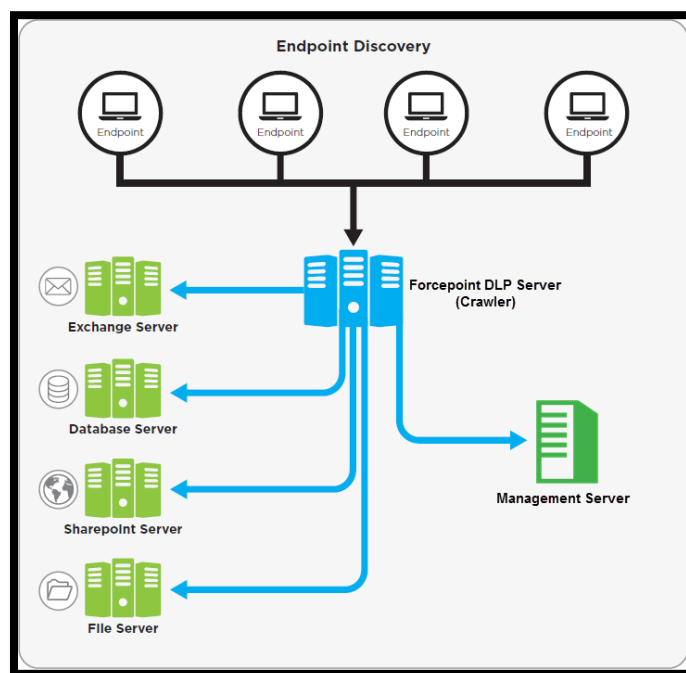


Figure 4 : Endpoint DLP at the endpoint level

Working Mechanism

The agent installed on the device integrates with the operating system at the kernel or user mode level. It monitors system activities, including the file system, clipboard, running applications, and connected devices. It can also apply policies locally even when the device is offline.

Endpoint Monitoring Components

Component	Description	Monitoring Examples
Peripheral Control	Controls connected devices and communication ports.	USB, Bluetooth, Wi-Fi Direct, printers, CD/DVD.
Application Control	Monitors and controls applications that access data.	Preventing chat applications from reading confidential files.
Clipboard Control	Monitors copy and paste operations.	Preventing sensitive data from being pasted into unauthorized applications.
Screen Capture	Prevents or records screenshot attempts.	Blocking Snip & Sketch when a confidential document is open.
Print Control	Controls printing operations.	Preventing or logging the printing of confidential documents.
File Transfer	Monitors file transfer between applications and locations.	Preventing a file from being copied from a protected folder to a public folder.

Table 7 : Endpoint monitoring components

Advantages

Advantage	Description
Offline protection	Continues working even when the device is outside the network.
Full peripheral control	Controls USB, Bluetooth, printers, and other devices.
Comprehensive user-activity visibility	Captures user activities that may not pass through the network.
Local data protection	Prevents malicious applications from accessing local files.

Table 8 : Advantages of Endpoint Monitoring Components

Challenges

Challenge	Description
Complex large-scale management	Updating and configuring thousands of devices requires strong infrastructure.
Device resource consumption	It may affect device performance if not efficiently designed.
Operating-system compatibility	It may face issues with operating system or application updates.

Possibility of agent removal	Users with high privileges may disable or uninstall the agent.
------------------------------	--

Table 9 : Challenges facing monitoring components on Endpoint

Suitable Use Cases

Endpoint DLP is suitable for:

- Organizations that rely on remote or field work.
- Environments with highly sensitive data, such as financial, governmental, and healthcare institutions.
- Controlling peripheral devices and preventing data leakage through USB.

2.4.1.3 Cloud DLP

Definition

Cloud DLP is specifically designed to protect data in cloud environments, whether SaaS applications such as Microsoft 365, Google Workspace, Salesforce, and Slack, or IaaS/PaaS platforms such as AWS, Azure, and GCP.

Types of Cloud DLP

Type	Description	Examples
Built-in SaaS DLP	DLP capabilities embedded within SaaS applications.	Microsoft Purview DLP, Google Workspace DLP.
CASB	A cloud access security broker that acts as an intermediate layer between users and cloud providers.	Netskope, McAfee MVISION Cloud, iBoss.
API-Based DLP	Integrates with SaaS applications through APIs to monitor stored data.	Monitoring SharePoint and Teams files through APIs.
Cloud Native DLP	DLP capabilities embedded in cloud infrastructure.	AWS Macie, Azure Information Protection.

Table 10 : Types of Cloud DLP

CASB Working Mechanism

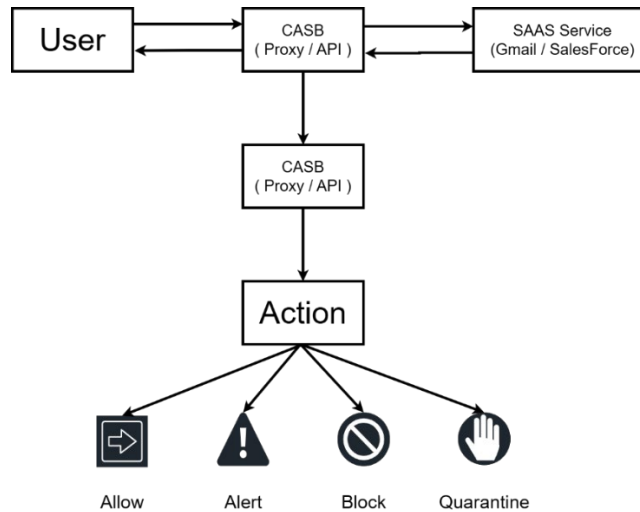


Figure 5 : CASB Working Mechanism

Advantages of CASB

Advantage	Description
SaaS coverage	Covers commonly used cloud applications such as Office 365, Gmail, and Slack.
No endpoint installation required	Works through APIs or proxies without user-device intervention.
Scalability	Adapts to the organization's growing use of cloud applications.
Shadow IT discovery	Detects unauthorized cloud applications used by employees.

Table 11 : CASB Advantages

Challenges of CASB

Challenge	Description
API dependency	Some applications do not provide sufficient APIs for comprehensive monitoring.
Data privacy	Passing data through a CASB may raise privacy concerns in some organizations.
Vendor diversity	Managing multiple cloud providers can complicate configuration and administration.
Additional cost	Most Cloud DLP solutions require additional subscriptions alongside SaaS subscriptions.

Table 12 : CASB Challenges

Suitable Use Cases

Cloud DLP is suitable for:

- Organizations that heavily rely on Microsoft 365 or Google Workspace.
- Hybrid work environments that depend on cloud applications.
- Organizations adopting a cloud-first strategy.

2.4.2 Classification Based on Protection Approach

Type	Description	Advantages	Disadvantages
Inline DLP	The system is placed directly in the data path, allowing violations to be prevented before they occur.	Immediate leakage prevention, minimal need for human intervention, reduced data-loss risk.	May introduce latency, requires high processing performance, and faces challenges with encryption.
API-Based DLP	Integrates with applications through APIs to monitor and analyze stored data asynchronously.	Does not affect user performance, covers data-at-rest, and integrates easily with SaaS applications.	Does not prevent violations in real time and depends on available APIs.
Hybrid DLP	Combines inline and API-based approaches to provide comprehensive protection.	Offers a balance between performance and protection, covering both data-in-motion and data-at-rest.	Higher cost and more complex management and configuration.

Table 13 : Types of protection methods

2.4.3 Classification Based on Data State

Data State	Description	Protection Mechanism	Example
Data-at-Rest	Data stored on servers, storage systems, databases, or user devices.	Scheduled scanning, automated classification, access control, and encryption.	Scanning SharePoint files to detect unprotected sensitive data.
Data-in-Motion	Data transmitted through the network, such as email, web traffic, FTP, or instant messaging.	Network DLP, packet inspection, and protocol analysis.	Inspecting email attachments before sending them outside the organization.
Data-in-Use	Data accessed or processed by applications or users.	Endpoint DLP, clipboard control, and application monitoring.	Preventing sensitive CRM data from being pasted into an unauthorized chat application.

Table 14 : Types of data that DLP handles

2.4.4 Classification Based on Vendor Source

DLP solutions can also be classified according to their source or vendor type.

Type	Description	Advantages	Challenges
Native OS DLP	DLP features embedded in operating systems or productivity platforms, such as Microsoft Purview or Google Workspace DLP.	Smooth integration, low additional cost, and easy management in homogeneous environments.	Limited features compared to specialized solutions and weak coverage of heterogeneous environments.
Integrated Security Platform DLP	DLP included as part of a broader security platform, such as Cisco Secure, Palo Alto Prisma, or Trellix.	Integration with firewalls, SIEM, and XDR, with centralized management.	May not be as specialized as standalone solutions and may create vendor ecosystem dependency.
Standalone DLP	Dedicated DLP solutions such as Forcepoint, Digital Guardian, or Symantec/Broadcom.	Advanced features, high flexibility, and strong support for complex environments.	Higher cost and requires integration with other systems for complete security visibility.
Open Source DLP	Open-source projects such as OpenDLP, Apache Spot, or MyDLP.	Low cost and high customization flexibility.	Limited support, irregular security updates, and requires strong technical expertise.

Table 15 : Classifying DLP solutions by source nature

2.4.5 Comprehensive Comparison of DLP Types

s	Network DLP	Endpoint DLP	Cloud DLP / CASB
Deployment location	Network gateway.	User device agent.	Between the user and the cloud.
Coverage	All devices on the network.	Only devices with the agent installed.	SaaS and cloud applications.
Offline protection	Not available.	Available.	Works through APIs.
USB and printer control	Not available.	Full control.	Not available.
Application monitoring	Limited to network traffic inspection.	Comprehensive.	Based on application API capabilities.
Performance impact	May introduce network delay.	May consume device resources.	Minimal API-related impact.
SIEM integration	High.	High.	High.
Suitable environment	Stable office environments.	Remote and field-work environments.	Cloud and hybrid-work environments.
Example solutions	Forcepoint Network DLP, Cisco Email Security.	Trellix Endpoint DLP, Digital Guardian.	Microsoft Purview, Netskope, iBoss.

Table 16 : General comparison between DLP types

2.4.6 Integrated Deployment Model

In modern organizations, relying on only one type of DLP is rarely sufficient. The optimal model is an integrated deployment that combines the three main types to provide comprehensive, multi-layered protection, also known as defense in depth.

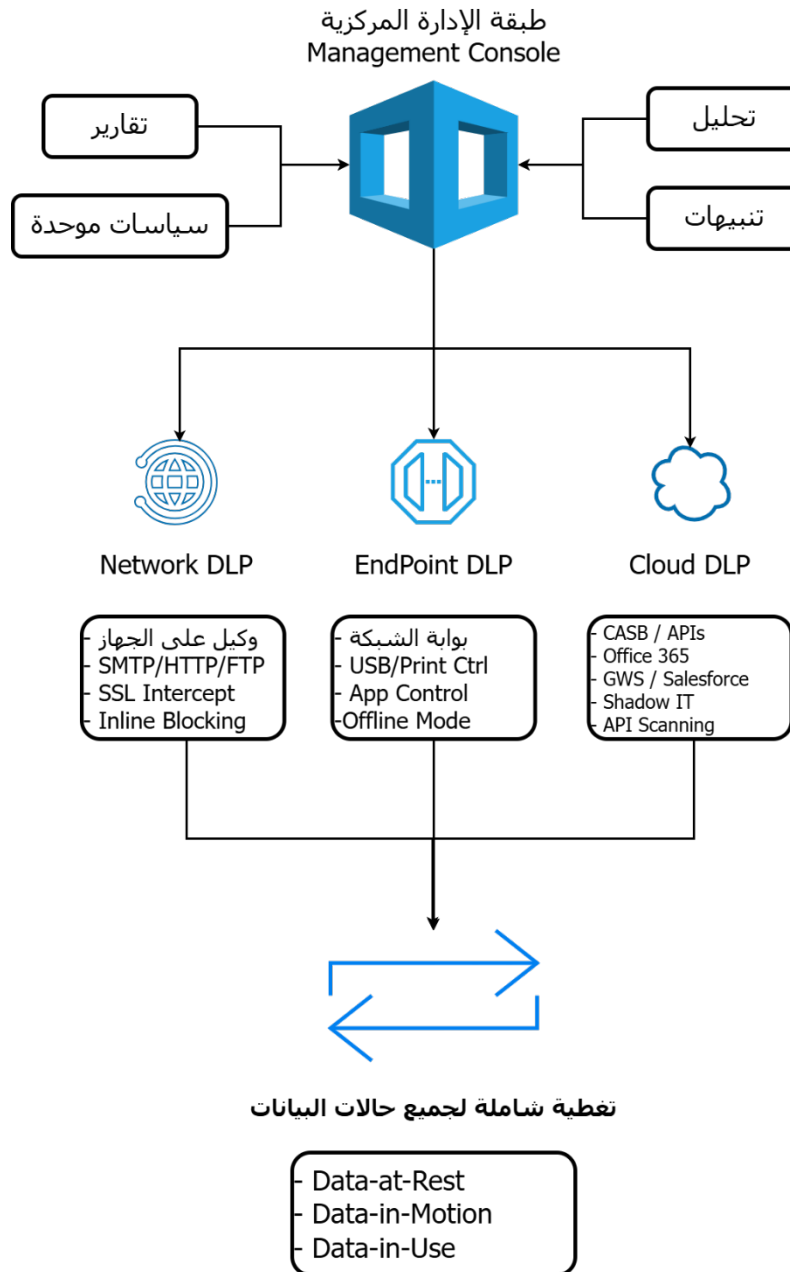


Figure 6 : General structure of the DLP system

Advantages of the Integrated Model

1. Comprehensive coverage:

Protects data in all states: stored, in motion, and in use.

2. Unified policy enforcement:

A single policy can be applied across all protection points.

3. Centralized visibility:

A unified dashboard displays all events and alerts.

4. Deployment flexibility:

Security layers can be added or removed according to organizational needs.

2.4.7 Criteria for Selecting the Appropriate DLP Type

When selecting the most suitable type of DLP for a project, several factors must be considered.

Criterion	Questions to Ask
Organizational structure	Does the organization rely on remote work? Are there unmanaged BYOD devices? Are cloud applications heavily used?
Nature of data	Is sensitive data centralized on servers or distributed across user devices? Is locally stored data protected?
Budget	Is there a budget for installing agents on every device? Is a SaaS subscription model preferred or an on-premises license?
Human resources	Is there a team capable of operating and managing the system? Is a managed service preferred or self-management?
Regulatory requirements	Are there compliance requirements such as HIPAA, GDPR, or PCI-DSS that require specific types of protection?

Table 17 : Factors influencing DLP selection

2.5 What Is a DLP System?

A DLP system is not merely a single software application. It is an integrated platform that combines software, policies, and workflows.

The system operates centrally, where the security administrator defines policies through a centralized management console. Distributed agents or sensors deployed across the network, servers, browsers, and endpoints then enforce these policies and report incidents.

The system also includes a centralized database for storing logs and analytical reports.

2.6 How DLP Works in Detail According to Deployment Environment

2.6.1 On Servers

DLP operates on servers by installing a software agent or integrating with the operating system to monitor the following:

- **Repositories:**

Scanning files stored on file servers, SharePoint, or databases to ensure that sensitive data is not stored in inappropriate locations.

- **Applications:**
Monitoring applications running on the server that handle sensitive data, such as ERP or CRM systems, to prevent unauthorized export.

2.6.2 On Browsers

Browser monitoring is performed through two main methods:

- **Proxy**
Analyzing outbound HTTPS traffic from the browser to detect attempts to upload sensitive files to platforms such as Gmail, Dropbox, or ChatGPT.
- **Browser Extensions**
In managed work environments, a DLP browser extension can be deployed to prevent copying and pasting sensitive data into unauthorized text fields or to block file uploads based on their content.

2.6.3 On Employee Endpoints

This is the most complex and control-intensive level. An agent installed on the user's operating system, such as Windows, macOS, or Linux, performs the following tasks:

- **Peripheral device control:**
Blocking or restricting the use of USB ports, Bluetooth, or printers.
- **Application monitoring:**
Controlling which applications may access sensitive data, such as preventing an unauthorized encrypted chat application from sending a file.
- **Screen capture monitoring:**
Recording the user's screen when a potential violation occurs in order to provide forensic evidence.
- **Offline mode:**
Continuing to enforce policies even when the device is not connected to the central network.

2.7 Capturing and Analyzing Internal Traffic

The role of DLP is not limited to monitoring external boundaries. It also extends to internal east-west traffic within the data center or internal network. This is achieved through:

- **SPAN ports or TAPs:**
Forwarding a copy of every packet passing through the network to the DLP analysis engine.
- **Deep Packet Inspection:**
Going beyond packet headers to inspect the actual data content, even if the data is compressed or partially encrypted, where TLS interception is applicable.
- **Behavioral analysis:**
Building a baseline for normal traffic behavior. For example, if a database server

suddenly establishes a connection to an external server at midnight and sends large amounts of data, this behavior is considered anomalous and is detected by the system.

2.8 Deep Content Inspection

Deep content inspection is the core of a DLP system. The system does not merely inspect the file extension or file name; it actually opens and analyzes the file content.

The techniques used include:

- **Digital Fingerprinting / Indexed Document Matching:**
Creating a unique digital fingerprint for sensitive documents, such as contracts or engineering drawings. Even if the file name is changed or some text is modified, the system can still identify the document.
- **Regular Expressions:**
Searching for specific patterns, such as credit card number formats.
- **Nested File Analysis:**
Inspecting compressed files such as ZIP and RAR, as well as attachments inside other attachments, to ensure that sensitive data is not hidden inside them.
- **Optical Character Recognition — OCR:**
Reading text inside images, such as screenshots containing financial data, and scanned documents.

2.9 Session Tracking and Reconstruction

To ensure accurate investigation of security incidents, advanced DLP systems reconstruct user sessions. Instead of viewing a single isolated log event, the system links related events together.

- **Session tracking:**
Linking all user activities within a specific time period, such as logging in, opening a document, copying it, pasting it into an email, and sending it.
- **Context:**
Understanding that the user copied sensitive data from a protected application and pasted it seconds later into a personal web browser.
- **Reconstruction:**
Providing an interface that allows the administrator to view the user's session as if they had been observing it directly, making it easier to distinguish between human error and malicious intent.

2.10 Immediate Alerting and Response Mechanisms

When a DLP policy violation is detected, the system performs a series of automated actions:

- 1. Alerting**
Sending an immediate alert to the Security Operations Center through email, SIEM integration, or the dashboard.
- 2. Blocking**
Completely preventing data transfer and displaying a message to the user explaining that the action violates policy.
- 3. Quarantine**
Isolating the email or file in a holding area until the administrator reviews it and manually approves or rejects it.
- 4. Encryption**
In certain cases, if sensitive data is being sent to a trusted external partner, the system automatically encrypts the email without interrupting the workflow.
- 5. User Education**
Displaying a pop-up window explaining to the user why the action was blocked and how to handle the data properly, helping improve user behavior over time.

2.11 Data Classification

Data classification is the cornerstone of any effective DLP system. The process includes:

- **Discovery:**
Identifying where sensitive data exists, whether on servers, in the cloud, or on user devices.
- **Labeling:**
Adding metadata labels to files, such as “Confidential,” “Public,” or “Personal.” Labeling can be performed manually by the user or automatically by the system based on content.
- **Hierarchical classification:**
Building a classification structure that reflects data sensitivity, such as:
Public → Internal → Confidential → Restricted
- Policies are then applied based on this classification.

2.12 Policy Enforcement

Policy enforcement is the practical implementation stage. A DLP policy is a logical rule consisting of:

- **Condition:**
Who is the user? What data is being handled? Where is it being sent? Which application or channel is being used?
- **Action:**
Monitor only, alert, block, quarantine, or allow with logging.
- **Exceptions:**
Defining cases where sensitive data transfer is permitted despite its sensitivity, such as sending data to an approved business partner's email address.

Policies are centrally deployed to ensure consistent security enforcement across all endpoints and gateways.

2.13 State of the Art

In recent years, DLP systems have evolved significantly to address new challenges such as hybrid work, end-to-end encryption, and generative artificial intelligence. Traditional rule-based or signature-based solutions are no longer sufficient.

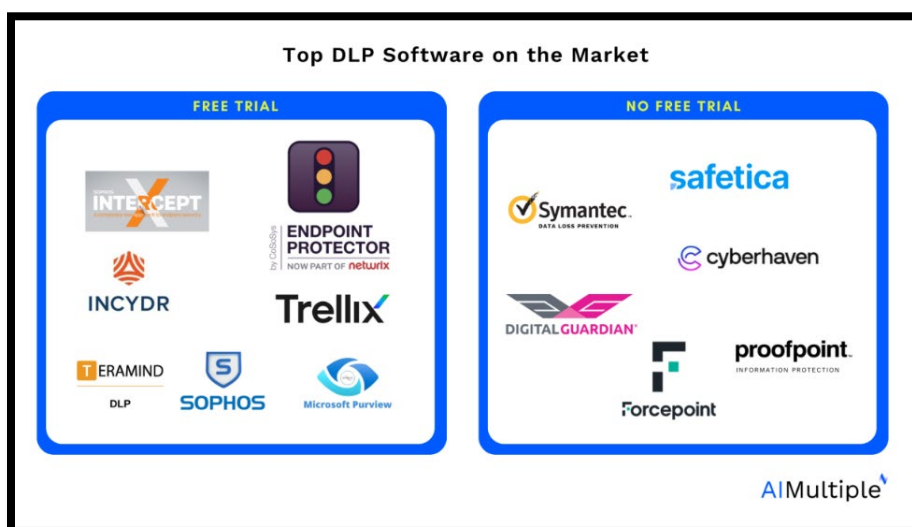


Figure 7 : Top-tier DLP brands on the market, based on pricing.

Recent Trends and Technologies

1. Integration of AI and Machine Learning

- **Intelligent contextual classification:**
Using NLP models to understand the meaning of text rather than merely searching for keywords. For example, the system can distinguish between a medical term in a scientific article and sensitive patient information.
- **User and Entity Behavior Analytics — UEBA:**
Building a behavioral model for each employee. For instance, if an employee usually sends 5 MB per day but suddenly attempts to send 5 GB at 3 a.m., the

system classifies the action as high risk, even if the data is not explicitly classified as sensitive.

2. DLP in TLS 1.3 and End-to-End Encryption Environments

Because intercepting encrypted HTTPS traffic has become increasingly difficult, modern solutions have shifted toward API-based integration with cloud applications, such as Microsoft 365 DLP and Google Workspace DLP, instead of relying only on network-level decryption.

3. Data Protection in Generative AI Environments

A new generation of DLP solutions has emerged to prevent engineers and employees from pasting source code or sensitive data into tools such as ChatGPT, Copilot, or Gemini, while still allowing these tools to be used safely through Shadow AI Discovery.

4. Integration of DLP with XDR and SIEM

DLP is moving from being a standalone solution to becoming a core component of Extended Detection and Response platforms. DLP events are linked with endpoint events and other incidents to provide unified visibility.

5. Multi-Cloud DLP

These solutions operate as a control layer above AWS, Azure, and GCP. They monitor configurations such as S3 buckets to prevent leaks caused by human misconfiguration.

Traditional DLP vs. Next-Generation DLP

Criterion	Traditional DLP	Next-Generation / State-of-the-Art DLP
Analysis methodology	Rule-based and regular-expression based.	AI/ML-based with contextual awareness.
Target environment	Centralized on-premises network perimeter.	Cloud-first, hybrid, and distributed endpoints.
Encryption handling	Relies on SSL interception, which is difficult with TLS 1.3.	Relies on APIs and native integrations.
Behavior detection	Limited and mainly content-focused.	Correlates behavior, content, and identity.
Response	Simple allow/block model.	Automated SOAR response, real-time user education, and automatic remediation.
New application support	Limited support for tools such as GenAI, Slack, and Teams.	Built-in support for SaaS and GenAI applications through APIs.

Table 18 : State of the Art

2.14 Reference Studies

To validate the requirements and understand practical challenges, several leading academic papers and industry reports were reviewed. These studies were selected to cover both theoretical and practical aspects of DLP systems.

Key Reference Studies

1. Gartner, “Market Guide for Data Loss Prevention” — 2022, renewed 2024

Summary:

This is considered one of the most important industry references. It identifies current trends, provides a vendor evaluation matrix, and emphasizes that DLP has become part of a broader strategy known as Data Security Platforms rather than a standalone product.

Key takeaway:

The proposed system should focus on seamless integration with cloud SaaS environments and support unified policies across endpoints and the network.

2. NIST Special Publication 800-175, “Guide to Data Loss Prevention” — 2020

Summary:

This methodological academic reference provides a theoretical framework for data classification and policy enforcement in governmental and private organizations.

Key takeaway:

It provides a strong foundation for data classification and highlights the importance of separating prevention policies from monitoring policies.

3. SANS Institute, “Practical Implementation of DLP: Avoiding the Pitfalls” — 2023

Summary:

This practical study focuses on common mistakes during DLP implementation, such as alert fatigue and endpoint performance impact. It presents a phased implementation approach that begins with monitor mode before enabling block mode.

Key takeaway:

The system should initially operate in a learning or monitoring mode to avoid business disruption.

4. Forrester Wave™: Data Loss Prevention Suites — 2024

Summary:

This analytical comparative report evaluates major DLP vendors such as Microsoft, Broadcom, and Trellix. The report focuses on the ability of solutions to handle hybrid work environments and unstructured data.

Key takeaway:

The ability to perform deep content inspection on unstructured files such as PDF, CAD files, and source code is a key differentiator between average and advanced solutions.

5. ACM Computing Surveys, “A Survey on Data Leakage Prevention Systems” — 2021**Summary:**

This comprehensive academic survey reviews algorithms used in leakage detection and compares techniques such as fingerprinting and NLP in terms of accuracy and false positives.

Key takeaway:

The study supports the use of hybrid analysis that combines fingerprinting for static documents and NLP for text communications in order to achieve higher accuracy.

Comparative Table of Reference Studies

Study	Year	Main Focus	Added Value to the Current Project
Gartner Market Guide	Gartner, 2022–2024	Market strategy, vendor analysis, and the future of DLP as part of DSP.	Highlights the need to support Cloud APIs and Unified Policy Management.
NIST SP 800-175	NIST, 2020	Theoretical framework, data classification, and risk management.	Provides a structured model for data classification and documentation mechanisms.
SANS Practical Implementation	SANS Institute, 2023	Practical deployment, avoiding mistakes, and monitor mode.	Supports a phased rollout approach: monitoring, simulation, then blocking.
Forrester Wave: DLP Suites	Forrester, 2024	Commercial product evaluation and handling unstructured data.	Emphasizes the need for deep content inspection of technical files, such as CAD and source code.
ACM Survey on DLP	ACM, 2021	Detection algorithms, fingerprinting, NLP, ML, accuracy, and false positives.	Supports hybrid analysis to balance scanning speed and detection accuracy.

Table 19 : Reference studies

2.15 Summary

Through the detailed review of fundamental concepts and reference studies, it becomes clear that building an effective DLP system requires more than a technical tool. It requires a comprehensive vision that begins with accurate data classification, continues through a deep understanding of traffic movement across different levels, including servers,

browsers, and endpoints, and ends with intelligent response mechanisms that do not disrupt workflow.

The state-of-the-art review shows that the industry is rapidly moving toward integrating artificial intelligence and User and Entity Behavior Analytics to address the challenges of encryption and remote work. Meanwhile, the reference studies from Gartner, NIST, and SANS emphasize the importance of cloud integration and gradual implementation to ensure project success.

Based on these foundations, the following sections of the project will move toward the analysis and design phase, where these theoretical concepts will be translated into specific functional requirements and a defined system architecture.

Chapter Three

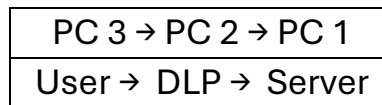
-Development and Execution-

3.1 Introduction

This chapter documents the practical development and execution process of the proposed web-based Data Loss Prevention system within an isolated local area network environment. While the previous chapters introduced the theoretical background of DLP systems, data classification, content inspection, behavioral analysis, and enforcement mechanisms, this chapter focuses on how these concepts were translated into a working laboratory implementation.

The purpose of this chapter is to describe the complete development workflow, starting from the preparation of the network environment and ending with the execution of the final DLP engine. The chapter covers the configuration of the three-machine LAN environment, the setup of the internal file server, the generation of representative company files, the deployment of MITMproxy as the interception point, the creation of user role profiles, the development of multiple DLP engine versions, and the training of the behavior and sensitivity AI models.

The implemented system was designed around a simple but controlled traffic path:



In this structure, PC 3 represents the domain user machine, PC 2 represents the DLP gateway and AI analysis point, and PC 1 represents the internal server that hosts the company files. Instead of allowing the user machine to communicate directly with the server, requests are routed through PC 2, where the DLP engine inspects the request, analyzes the requested file content, evaluates the user behavior profile, and then decides whether the interaction should be allowed, logged, warned, or blocked.

A key design decision in this project was to separate **AI scoring** from **policy enforcement**. The AI models do not directly decide whether a request is allowed or blocked. Instead, the sensitivity model produces content-related scores, while the behavior model produces profile-related risk scores. The final decision is made by the DLP engine using a weighted risk equation and a set of enforcement rules. This separation improves explainability and makes the system easier to evaluate and justify.

The development process followed an incremental approach. First, the basic network communication between the three machines was verified. Then, an internal HTTP file server was deployed on PC 1. After that, MITMproxy was configured on PC 2 to confirm that requests from PC 3 could be intercepted and forwarded correctly. Once the interception path was verified, the DLP engine was developed gradually, starting with a simple logging version and progressing toward an AI-assisted version that integrates both behavior and sensitivity analysis.

This chapter also includes the execution evidence required to validate the system. Screenshots, command outputs, and generated reports are used to demonstrate that the system was successfully executed in the laboratory environment. Tables are also included

to summarize the role of each device, the generated test files, the supported user actions, the AI model outputs, and the final DLP decisions.

The final implementation demonstrates that the proposed system can perform the following tasks:

- Intercept file interaction requests from a user machine.
- Analyze the content of requested files.
- Classify the sensitivity level of the content.
- Compare the requested action against the selected user role profile.
- Calculate a final risk score.
- Apply an enforcement decision: ALLOW, LOG, WARN, or BLOCK.
- Store decisions in structured logs.
- Export the results into CSV reports for analysis.

Therefore, this chapter serves as the practical documentation of the system implementation and provides the technical evidence that the proposed architecture was successfully developed and executed.

3.2 Laboratory Environment

The proposed system was implemented in an isolated LAN environment consisting of three Windows-based machines connected to the same network switch. The laboratory environment was intentionally isolated from the Internet to ensure that all traffic, files, and AI models remained local to the experiment. This design allowed the DLP system to be tested under controlled conditions without relying on external cloud services or online APIs.

The network was designed to simulate a small internal organization where users interact with internal files hosted on a company server. In this setup, PC 1 acts as the internal server, PC 2 acts as the DLP gateway, and PC 3 acts as the user endpoint. The purpose of this topology is to force all file interaction requests from the user machine to pass through the DLP gateway before reaching the internal server.

The laboratory topology is shown conceptually below:

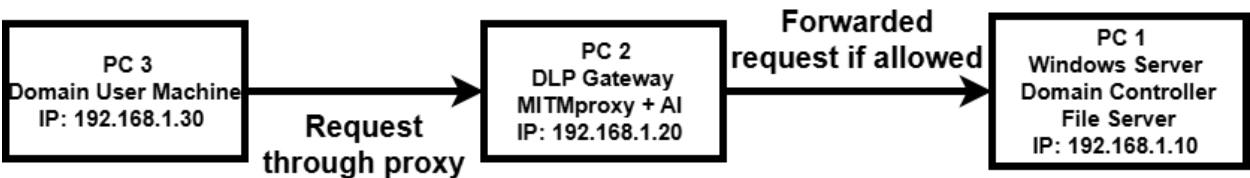


Figure 8 : Lab- Topology

Device	Role	Main Function	IP Address
PC 1	Internal Server	Windows Server, Domain Controller, and file server	192.168.1.10
PC 2	DLP Gateway	MITMproxy, DLP Engine, Behavior AI, and Sensitivity AI	192.168.1.20

PC 3	User Endpoint	Domain-joined Windows client used to simulate employee actions	192.168.1.30
------	---------------	--	--------------

Table 20 : Laboratory Devices

PC 1 was configured as the internal server of the laboratory. It was promoted to a domain controller and was also used to host the company test files. A lightweight Python HTTP server was later deployed on this machine to serve internal documents to the user endpoint.

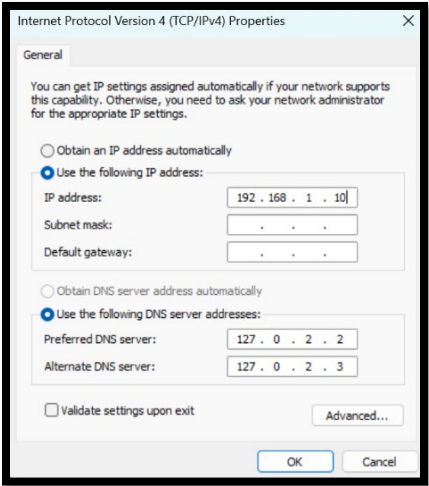


Figure 9 : Server Static IP Assigning

PC 2 was configured as the central DLP processing point. It was not joined to the domain because its role was to act as an independent inspection and enforcement gateway. MITMproxy was installed on this machine to intercept HTTP requests, and all DLP engine versions and AI models were developed and executed on it.

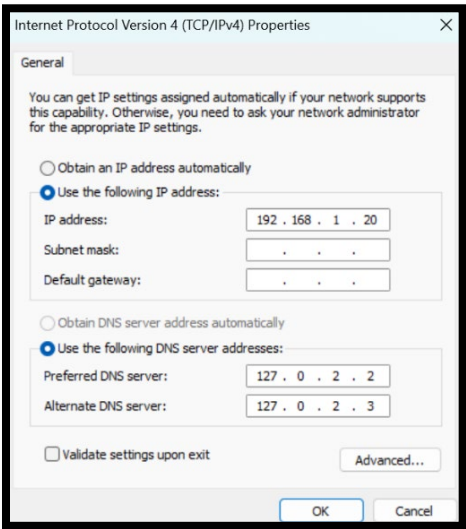


Figure 10 : D.L.P Static IP Assigning

PC 3 was configured as the normal user machine. It was joined to the domain and used to simulate employee activity. All test requests were launched from this machine and routed through PC 2 using proxy settings or explicit curl proxy commands.

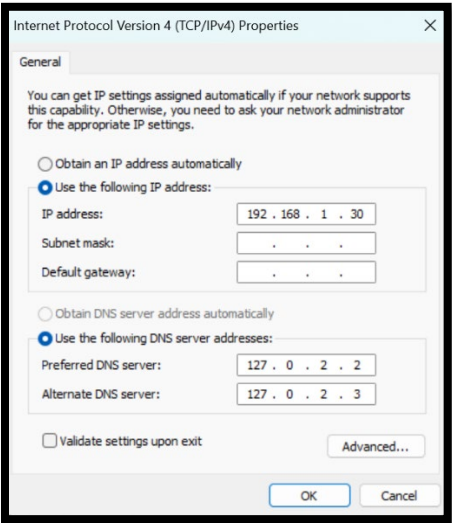


Figure 11 : Normal User PC IP

The main reason for selecting this topology is that it provides a clear and testable DLP flow. The server does not directly serve the user without passing through PC 2. This makes PC 2 the enforcement point where the system can analyze the content, evaluate the user action, and determine whether the interaction should continue.

Characteristic	Description
Network Type	Isolated LAN
Internet Access	Disabled / not required
Addressing	Static IP addresses
Traffic Direction	PC 3 → PC 2 → PC 1
Server Role	Internal document hosting
DLP Role	Interception, analysis, scoring, decision, and logging
AI Execution	Local execution on PC 2
Test Method	HTTP requests using curl through MITMproxy

Table 21 : Laboratory Characteristics

3.3 Network Configuration and Connectivity Verification

Before deploying the DLP engine, the basic network configuration had to be verified. Since the laboratory depends on a three-machine communication path, each device was assigned a static IP address. This ensured that the file server, the DLP gateway, and the user machine could communicate consistently throughout the development and testing process.

The static addressing scheme used in the laboratory was as follows:

Device	IP Address	Purpose
PC 1	192.168.1.10	Internal file server and domain controller
PC 2	192.168.1.20	DLP gateway and proxy listener
PC 3	192.168.1.30	User endpoint and test client

Table 22 : Static IP Addressing Scheme

After assigning the static IP addresses, basic connectivity tests were performed using the ping command. These tests were necessary to confirm that the three machines were reachable before configuring the file server and MITMproxy.

The first test verified that PC 3 could reach PC 1 & PC 2:

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator>ping 192.168.1.20

Pinging 192.168.1.20 with 32 bytes of data:
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\Administrator>ping 192.168.1.30

Pinging 192.168.1.30 with 32 bytes of data:
Reply from 192.168.1.30: bytes=32 time<1ms TTL=128
Reply from 192.168.1.30: bytes=32 time<1ms TTL=128
Reply from 192.168.1.30: bytes=32 time<1ms TTL=128
Reply from 192.168.1.30: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.30:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Figure 12 : Ping 1

The second test verified that PC 2 could reach PC 3 & PC 1:

```
Microsoft Windows [Version 10.0.19045.2846]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

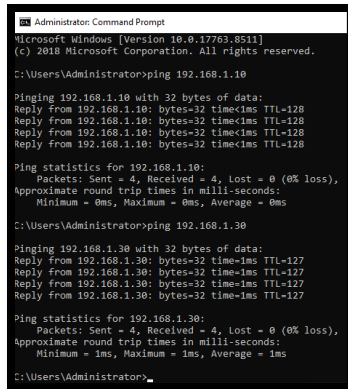
C:\Windows\system32>
C:\Windows\system32>
C:\Windows\system32>ping 192.168.1.20

Pinging 192.168.1.20 with 32 bytes of data:
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Figure 13 : Ping 2

The third test verified that PC 1 could reach PC 2 & PC 3 :



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.17763.8511]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\Administrator>ping 192.168.1.30

Pinging 192.168.1.30 with 32 bytes of data:
Reply from 192.168.1.30: bytes=32 time=1ms TTL=127
Reply from 192.168.1.30: bytes=32 time=1ms TTL=127
Reply from 192.168.1.30: bytes=32 time=1ms TTL=127
Reply from 192.168.1.30: bytes=32 time=1ms TTL=127

Ping statistics for 192.168.1.30:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 1ms, Average = 1ms

C:\Users\Administrator>
```

Figure 14 : Pingging 3

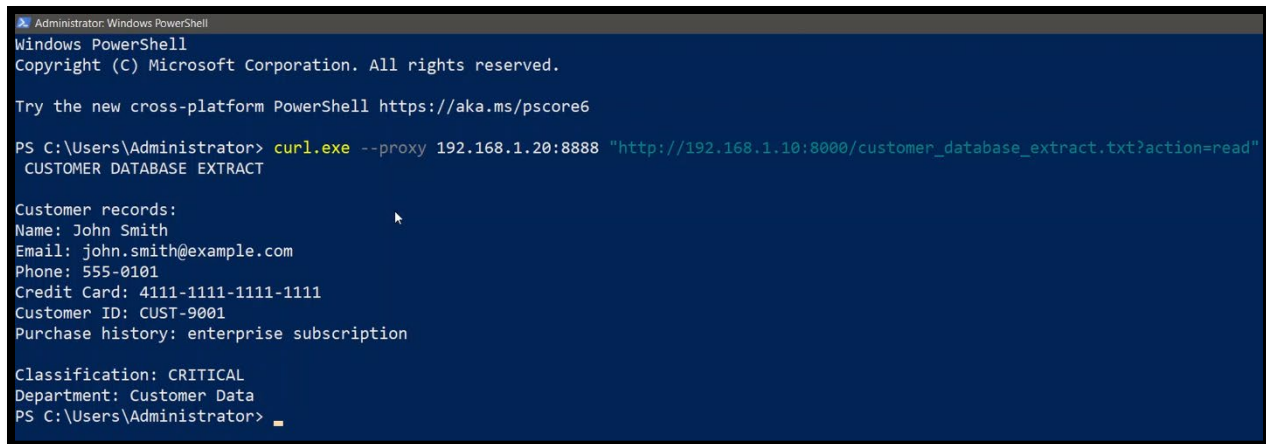
Successful replies from these tests confirmed that the three machines were correctly connected to the same local network and that there were no basic Layer 3 connectivity issues.

After confirming ICMP connectivity, the next verification step was to test whether PC 3 could reach the internal file server hosted on PC 1. This was later done using HTTP requests after the Python file server was deployed. The successful network verification established the foundation for the remaining stages of the project, because MITMproxy and the DLP engine both depend on stable communication between the endpoint, gateway, and server.

At this stage, the traffic was not yet forced through PC 2 at the network gateway level. Instead, the test traffic was explicitly sent through PC 2 using proxy commands. This approach was selected to simplify debugging and to ensure that each development milestone could be validated separately before moving to full enforcement.

The general test method used later in the project followed this format:

`curl.exe --proxy 192.168.1.20:8888 http://192.168.1.10:8000/file_name.txt`



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Administrator> curl.exe --proxy 192.168.1.20:8888 "http://192.168.1.10:8000/customer_database_extract.txt?action=read"
CUSTOMER DATABASE EXTRACT

Customer records:
Name: John Smith
Email: john.smith@example.com
Phone: 555-0101
Credit Card: 4111-1111-1111-1111
Customer ID: CUST-9001
Purchase history: enterprise subscription

Classification: CRITICAL
Department: Customer Data
PS C:\Users\Administrator>
```

Figure 15 : Successful CURL Command

This command sends the request from PC 3 to PC 2 first, where MITMproxy and the DLP engine inspect the traffic. If the request is allowed, PC 2 forwards it to PC 1. This confirms the intended traffic path: PC 3 → PC 2 → PC 1

3.4 File Server Setup on PC 1

After verifying that the three machines could communicate over the isolated LAN, the next step was to configure PC 1 as the internal file server. In the laboratory topology, PC 1 represents the organization's internal server that stores documents belonging to different departments such as Engineering, HR, Finance, Legal, Sales, IT/Security, and Management.

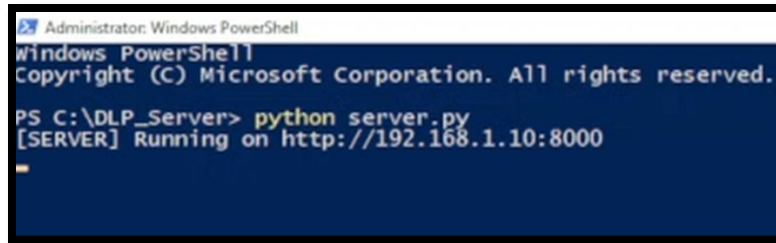
A lightweight Python HTTP server was used instead of a full web server such as IIS or Apache. This approach was selected because it provides a simple and controllable environment for testing file access operations. It also allows the server-side behavior to be customized easily, such as printing request logs and supporting query parameters used later to simulate different file actions.

- The server was placed in the directory: C:\DLP_Server
- The Python server script was named: server.py

The server listens on port 8000 instead of port 80. This was done to avoid permission issues and conflicts with existing Windows services. During the initial setup, port 80 caused a socket permission error, so port 8000 was selected as a more stable option for the lab.

The server was started using the following command on PC 1:

- `cd C:\DLP_Server`
- `python server.py`

A screenshot of a Windows PowerShell console window titled "Administrator: Windows PowerShell". The window shows the following text: "Windows PowerShell", "Copyright (C) Microsoft Corporation. All rights reserved.", "PS C:\DLP_Server> python server.py", and "[SERVER] Running on http://192.168.1.10:8000".

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\DLP_Server> python server.py
[SERVER] Running on http://192.168.1.10:8000
```

Figure 16 : Server Initialization

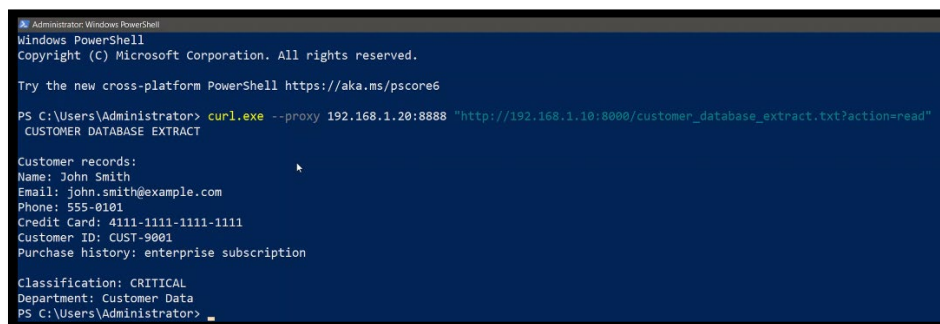
The server was designed to process HTTP GET requests and serve files from the C:\DLP_Server directory. It also logs each request in the PowerShell console, which was useful during debugging and validation.

A key modification was added to the server to support action simulation through query parameters. For example, the DLP test requests may contain:

- ?action=read
- ?action=delete
- ?action=upload_share

Without special handling, the server would treat the full URL path including the query string as a filename. To avoid this issue, the server uses URL parsing to separate the actual file path from the query string. This allows the server to open the correct file while still displaying the requested action for logging.

Example request:

A screenshot of a Windows PowerShell console window titled "Administrator: Windows PowerShell". The window shows the following text: "Windows PowerShell", "Copyright (C) Microsoft Corporation. All rights reserved.", "Try the new cross-platform PowerShell https://aka.ms/pscore6", "PS C:\Users\Administrator> curl.exe --proxy 192.168.1.20:8888 "http://192.168.1.10:8000/customer_database_extract.txt?action=read"", "CUSTOMER DATABASE EXTRACT", "Customer records:", "Name: John Smith", "Email: john.smith@example.com", "Phone: 555-0101", "Credit Card: 4111-1111-1111-1111", "Customer ID: CUST-9001", "Purchase history: enterprise subscription", "Classification: CRITICAL", "Department: Customer Data", and "PS C:\Users\Administrator>".

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Administrator> curl.exe --proxy 192.168.1.20:8888 "http://192.168.1.10:8000/customer_database_extract.txt?action=read"
CUSTOMER DATABASE EXTRACT

Customer records:
Name: John Smith
Email: john.smith@example.com
Phone: 555-0101
Credit Card: 4111-1111-1111-1111
Customer ID: CUST-9001
Purchase history: enterprise subscription

Classification: CRITICAL
Department: Customer Data
PS C:\Users\Administrator>
```

Figure 17 : Example request

Expected server output:

```

Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\DLP_Server> python server.py
[SERVER] Running on http://192.168.1.10:8000
[SERVER] READ /customer_database_extract.txt
[SERVER] QUERY action=read

```

Figure 18 : Expected Server Output

This behavior was important because the DLP engine uses the query parameter to simulate different actions while the server still returns the correct file content.

Item	Value
Server machine	PC 1
Server IP	192.168.1.10
Server directory	C:\DLP_Server
Server script	server.py
Protocol	HTTP
Port	8000
Main purpose	Hosting internal company files

Table 23 : File Server Configuration

3.5 Company File Generation

After preparing the file server, a set of representative company files was generated on PC 1. These files were created to simulate the type of documents commonly found inside an organization. The goal was not to test file extensions, but to test the system’s ability to analyze the actual content of each file and determine its sensitivity level.

- All files were generated inside: C:\DLP_Server
- A PowerShell script was used to automatically create the test files:
generate_company_files.ps1

The generated files covered several departments and data categories, including public internal documents, engineering documents, legal contracts, HR records, finance reports, sales documents, IT/security files, credentials, customer data, and personal information.

The DLP system later used these files to test whether the sensitivity model could detect the category and sensitivity level from content rather than relying on the file name or extension. This was important because the project objective is to avoid extension-based detection and focus on content and behavior.

Category	Example File	Expected Sensitivity
Public/Internal General	general_company_policy.txt	LOW
Public/Internal General	public_holiday_schedule.txt	LOW

Engineering	engineering_project_alpha.txt	MEDIUM
Engineering	engineering_backend_design.txt	MEDIUM
Engineering	engineering_api_notes.txt	MEDIUM
Legal	legal_client_contract.txt	HIGH
Legal	legal_lawsuit_summary.txt	HIGH
Legal	legal_nda_template.txt	HIGH
HR	hr_employee_records.txt	HIGH
HR	hr_leave_policy.txt	MEDIUM
HR	hr_performance_reviews.txt	HIGH
Finance	finance_payroll_report.txt	HIGH
Finance	finance_budget_forecast.txt	HIGH
Finance	finance_bank_accounts.txt	CRITICAL
Sales	sales_customer_pipeline.txt	MEDIUM
Sales	sales_pricing_strategy.txt	HIGH
Sales	sales_leads_list.txt	HIGH
IT/Security	it_network_inventory.txt	MEDIUM
IT/Security / Credentials	it_admin_credentials_note.txt	CRITICAL
IT/Security	it_security_incident_report.txt	HIGH
Management	management_strategy_roadmap.txt	HIGH
Management	management_merger_plan.txt	CRITICAL
Customer Data	customer_database_extract.txt	CRITICAL
Personal Information	personal_identity_records.txt	CRITICAL

Table 24 : Company Files

The content of each file included keywords and phrases representing its business category

- Engineering files included terms such as: backend architecture, API design, source code, deployment, database schema
- Finance files included terms such as: salary, payroll, bonus, tax, bank transfer, budget, revenue, invoice
- Credential-related files included terms such as: username, password, token, api_key, secret, domain_admin
- Personal information files included terms such as: SSN, passport number, national ID, home address

These contents were later used by the sensitivity analysis component to classify files and assign content risk scores.

Purpose	Description
Content testing	Provides text content for the sensitivity model
Behavior testing	Allows role/action comparisons against different categories
Critical data simulation	Provides credentials, customer data, and personal identifiers
Reporting	Produces meaningful logs for the final decision report

Demo scenarios	Supports clear ALLOW, WARN, LOG, and BLOCK examples
----------------	---

Table 25 : Purpose of Files

3.6 MITMproxy Setup on PC 2

After preparing the file server on PC 1, PC 2 was configured as the interception and analysis point. The purpose of PC 2 is to receive file interaction requests from PC 3, inspect them using the DLP engine, and forward only the allowed requests to PC 1.

MITMproxy was selected as the interception tool because it can operate as an HTTP proxy and allows Python scripts to inspect and modify requests and responses. In this project, MITMproxy was used with mitmdump, which is the command-line interface suitable for scripted DLP processing.

MITMproxy was installed and executed on PC 2 using a listening port of 8888. The following command was used:

```
mitmdump -s test_logger.py --listen-host 0.0.0.0 --listen-port 8888
```

The parameter `--listen-host 0.0.0.0` was important because it allowed PC 2 to accept proxy connections from other machines on the LAN, not only from localhost. The parameter `--listen-port 8888` defined the proxy port used by PC 3.

When MITMproxy starts successfully, it displays a message indicating that the proxy server is listening on the configured port.

Item	Value
Machine	PC 2
IP address	192.168.1.20
Proxy software	MITMproxy / mitmdump
Listening host	0.0.0.0
Listening port	8888
Script mode	Python addon script
Initial test script	test_logger.py

Table 26 : MITMproxy Configuration

A simple logging script named `test_logger.py` was first used to verify that PC 2 could capture HTTP requests. The purpose of this script was not to enforce DLP decisions yet, but only to print request information such as the method, host, port, path, and URL.

When PC 3 sent a proxied request, the MITMproxy console displayed captured request details similar to the following:

Before testing file access through the proxy, the connectivity from PC 3 to the MITMproxy port was verified using: *Test-NetConnection 192.168.1.20 -Port 8888*

The expected result was: *TcpTestSucceeded: True*

Verification Point	Expected Result
PC 3 to PC 2 port 8888	TCP connection succeeds
PC 3 proxied curl request	File content is returned
PC 2 MITMproxy console	Request details are captured
PC 1 server console	Requested file is logged

Table 27 : Proxy Verification Results

3.7 Verifying the Traffic Path

After configuring MITMproxy on PC 2, the next step was to verify that user requests from PC 3 were successfully routed through PC 2 before reaching PC 1. This verification was essential because the DLP system depends on PC 2 being positioned logically between the user endpoint and the internal file server.

The traffic path used in the experiment was: PC 3 → PC 2 → PC 1

PC 3 generated HTTP requests using curl.exe, while PC 2 received the requests through MITMproxy on port 8888. If the request was allowed, MITMproxy forwarded it to the internal file server on PC 1.

The following command was used from PC 3 to test the proxy path:

```
curl.exe --proxy 192.168.1.20:8888 "http://192.168.1.10:8000/dashboard.html"
```

The expected results were observed on all three machines:

Machine	Expected Output
PC 3	The content of dashboard.html is displayed
PC 2	MITMproxy displays the captured HTTP request
PC 1	The server console prints [SERVER] READ /dashboard.html

This confirmed that PC 2 was successfully acting as the traffic interception point. The result also confirmed that the DLP system could later inspect requests and responses before they reached the user or the server.

3.8 User Profile Preparation

The system was designed to evaluate file interactions based on user role behavior. Therefore, a set of user profiles was prepared on PC 2. These profiles represent common employee roles inside an organization and were later used by the behavior AI model to evaluate whether a requested action matches the expected behavior of the selected role.

The supported roles were:

- Engineer
- Lawyer
- HR

- Finance
- Manager
- IT Admin
- Sales
- Normal Employee

A profile selection script was created on PC 2: *select_profile.py*

The script allows the operator to choose an active user and role before running the DLP engine. The selected profile is saved into: *active_profile.json*

Example output:

```
{
  "user": "Ahmed",
  "role": "Engineer"
}
```

The active profile is loaded by the DLP engine at startup. This allows the same file and action to produce different results depending on the selected role.

For example:

Role	File	Action	Expected Behavior
Engineer	Engineering file	read	Normal
Engineer	Finance payroll file	read	Unusual or risky
HR	Employee records	read	More normal
Normal Employee	Credentials file	read	Highly risky

The use of role profiles was not implemented as a fixed permission matrix. Instead, the role was used as an input feature for the behavior model, which calculates a profile match score and behavior risk score.

```
=====
DLP ACTIVE PROFILE SELECTOR
=====
Enter username, example Ahmed: Rami

Select role/profile:
1. Engineer
2. Lawyer
3. HR
4. Finance
5. Manager
6. IT Admin
7. Sales
8. Normal Employee

Enter choice number: 5

Active profile saved:
{
  "user": "Rami",
  "role": "Manager"
}

File created: active_profile.json
```

Figure 19 : user profiler selector

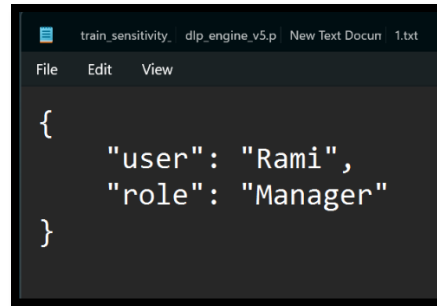


Figure 20 : Active profile

3.9 DLP Engine Architecture

The final DLP engine was designed as the central decision-making component of the proposed system. It runs on PC 2, which acts as the inspection and enforcement point between the user machine and the internal file server. The engine receives intercepted HTTP traffic from MITMproxy, extracts the required request information, analyzes the file content, evaluates the selected user profile, calculates a final risk score, and applies the appropriate enforcement action.

The final engine architecture is based on modular components. Each component performs a specific task and passes its output to the next stage. This modular design improves maintainability and makes the system easier to test and extend.

The main components of the final DLP engine are:

- MITMproxy interception layer
- Request and action detection module
- Sensitivity AI component
- Behavior AI component
- Risk scoring module
- Enforcement policy module
- Logging and reporting module

The general processing flow is shown below:

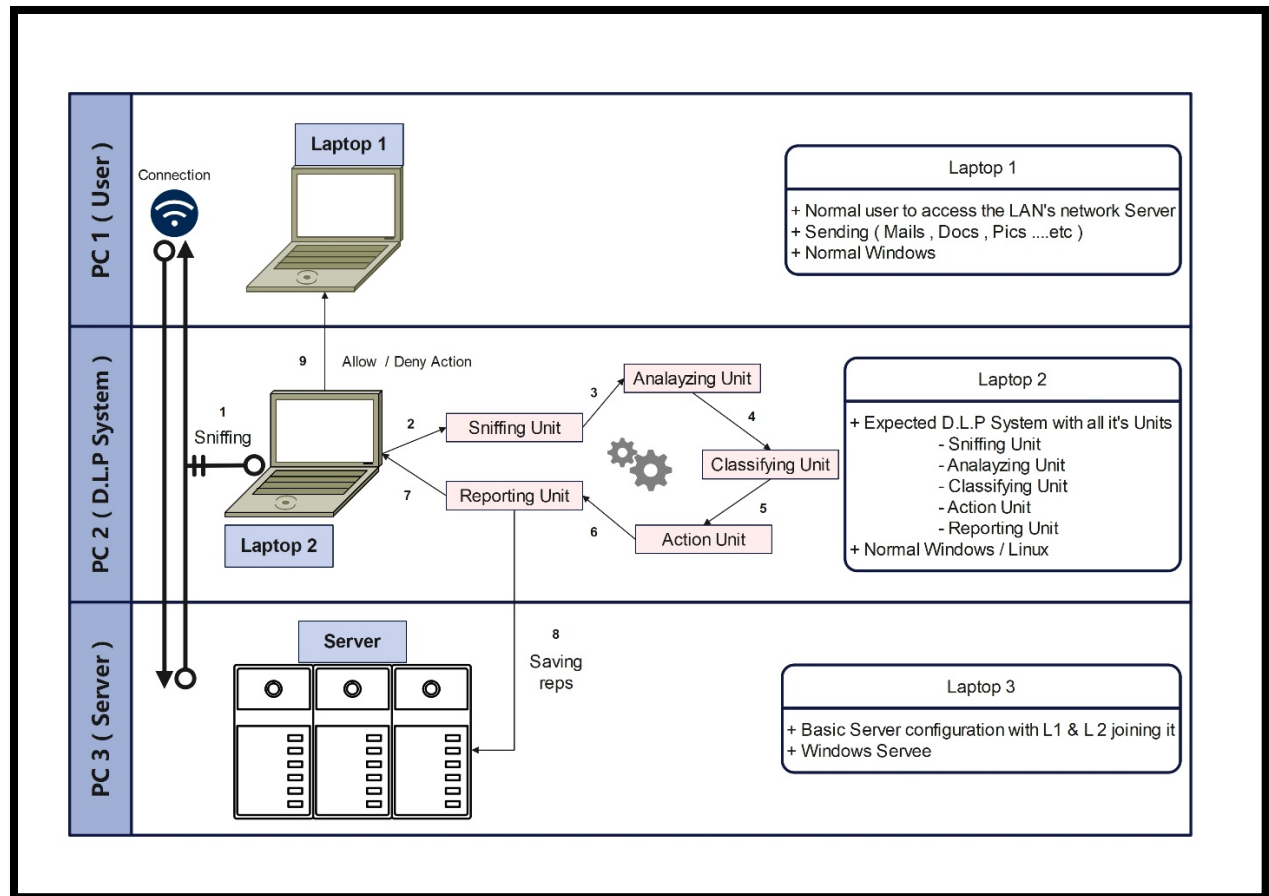


Figure 21 : processing flow

The engine does not rely on file extensions or network-flow features such as packet size, destination port, or traffic volume. Instead, it focuses on three main factors:

- The content of the requested file
- The selected user role/profile
- The requested action

This design matches the project goal of building a content-aware and profile-aware DLP system. The file content is analyzed by the Sensitivity AI, while the user behavior context is analyzed by the Behavior AI. The final enforcement decision is not made directly by either AI model. Instead, both models output numerical scores, and the DLP engine combines these scores using a defined risk equation.

The final architecture separates detection from enforcement. This separation is important because it makes the system explainable. The AI models produce risk indicators, but the DLP engine remains responsible for applying the final policy.

3.10 Request Processing and Action Detection

The request processing module is responsible for extracting the necessary information from each intercepted request. When PC 3 sends a request through PC 2, MITMproxy passes the request and response data to the DLP engine. The engine then extracts the requested file name, the selected action, the active user profile, and the file content.

The supported actions in the system are:

- read
- download
- write/edit
- delete
- upload/share

During the controlled laboratory testing, actions were simulated using URL query parameters. This approach allowed the same HTTP file server to be used for testing multiple file interaction types without implementing a full document management system. For example, the following request simulates a read action:

`curl.exe --proxy 192.168.1.20:8888`

`"http://192.168.1.10:8000/engineering_project_alpha.txt?action=read"`

The following request simulates a delete action:

`curl.exe --proxy 192.168.1.20:8888`

`"http://192.168.1.10:8000/engineering_project_alpha.txt?action=delete"`

Although the HTTP method remains GET during simulation, the DLP engine reads the action parameter and treats the interaction according to the requested action. This allowed the system to test different DLP decisions such as allowing a read action while blocking a delete or upload/share action for the same file.

The action detection module maps the query parameter into the internal action format used by the engine:

Query Parameter	Internal Action
action=read	read
action=download	download
action=write_edit	write/edit
action=delete	delete
action=upload_share	upload/share

Table 28 : action table

If no action parameter is found, the engine falls back to the HTTP method. For example, a normal GET request is treated as a read action.

The action is important because each action has a different level of risk. Reading a document is less dangerous than deleting or uploading it. Therefore, the detected action is later used in the risk scoring equation.

3.11 Sensitivity AI Component

The Sensitivity AI component analyzes the actual text content of the requested file. Its purpose is to determine the business category of the content, estimate its sensitivity level, and produce a numerical content risk score.

This component is essential because the system does not depend on file extensions. A file is not considered sensitive because it is a .txt, .pdf, or .csv file. Instead, the content itself is inspected and classified.

The Sensitivity AI receives the file text as input and produces the following outputs:

- category
- sensitivity_level
- content_risk_score
- category_confidence_score
- level_confidence_score

The supported content categories include:

- Public/Internal
- GeneralEngineering
- Legal
- HR
- Finance
- Sales
- IT/Security
- Management
- Customer Data
- Credentials/Secrets
- Personal Information
- Unknown

The supported sensitivity levels are:

- NONE
- LOW
- MEDIUM
- HIGH
- CRITICAL

3.11.1 Sensitivity Dataset

A synthetic content sensitivity dataset was generated to train the Sensitivity AI. The dataset contained text samples representing different organizational document categories. Each sample was labeled with a category, a sensitivity level, and a numerical content risk score.

The dataset file was: content_sensitivity_dataset.csv

The dataset Features were:

Column	Description
file_text	Text content sample
category	Business category of the content
sensitivity_level	Sensitivity label
content_risk_score	Numeric score between 0 and 1

Table 29 : dataset features

Example Text	Category	Sensitivity	Risk
backend architecture API deployment database schema	Engineering	MEDIUM	0.50
salary payroll bonus tax bank transfer	Finance	HIGH	0.75
password token api_key secret root	Credentials/Secrets	CRITICAL	1.00
contract agreement lawsuit clause	Legal	HIGH	0.75
holiday schedule office meeting policy	Public/Internal General	LOW	0.25
SSN passport national ID home address	Personal Information	CRITICAL	1.00

Table 30 : dataset features -2 example

3.11.2 Feature Extraction Using TF-IDF

Before training the model, the text data was converted into numerical features using TF-IDF vectorization.

TF-IDF stands for Term Frequency–Inverse Document Frequency. It assigns higher importance to terms that are frequent in a specific document but less common across the whole dataset. This is useful for text classification because it helps the model identify meaningful words that distinguish one category from another.

For example: "password", "token", "api_key" are highly meaningful for the Credentials/Secrets category,

while: "contract", "agreement", "lawsuit" are meaningful for the Legal category.

The vectorizer file generated during training was: sensitivity_vectorizer.pkl

3.11.3 Trained Models

The Sensitivity AI uses three trained models:

Model File	Algorithm	Purpose
sensitivity_category_model.pkl	Random Forest Classifier	Predicts content category
sensitivity_level_model.pkl	Random Forest Classifier	Predicts sensitivity level
sensitivity_risk_model.pkl	Random Forest Regressor	Predicts numeric content risk

Table 31 : models

The label encoders were saved in: sensitivity_label_encoders.pkl

The category and sensitivity level are classification tasks, while the content risk score is a regression task.

3.11.4 Sensitivity AI Workflow

The Sensitivity AI works as follows:

1. The DLP engine extracts the file text from the HTTP response.
2. The text is passed to the TF-IDF vectorizer.
3. The category classifier predicts the content category.
4. The sensitivity classifier predicts the sensitivity level.
5. The risk regressor predicts the content risk score.
6. The engine applies a critical keyword safety override if necessary.
7. The final sensitivity outputs are passed to the Behavior AI and risk engine.

3.11.5 Critical Keyword Safety Override

Although the trained Sensitivity AI is the main classification component, the final engine also includes a critical keyword safety override. This override is used only for highly sensitive patterns that should not be missed.

Examples include:

- Password
- Token
- api_key
- secret
- SSN
- Passport
- credit card
- national ID

If such patterns are detected, the system can force the category to a critical category such as:

- Credentials/Secrets
- Personal Information
- Customer Data

and raise the sensitivity level to: **CRITICAL**

This safety override does not replace the Sensitivity AI. Instead, it provides an additional protection layer for high-risk data.

3.12 Behavior AI Component

The Behavior AI component evaluates whether a requested file interaction is normal for the selected user role. It does not inspect file text directly. Instead, it receives the content

category and sensitivity level from the Sensitivity AI, along with the selected user role and requested action.

The Behavior AI input is:

- Role
- Action
- Category
- Sensitivity_Level

The Behavior AI output is:

- profile_match_score
- behavior_risk_score

The model files used by the Behavior AI are:

- behavior_risk_model.pkl
- behavior_score_encoders.pkl

3.12.1 Role-Based Behavior Concept

Each role has a different expected behavior pattern. For example, an Engineer is expected to interact with Engineering documents more often than Finance or HR files. Similarly, HR users are more likely to interact with employee records, while Finance users are more likely to interact with payroll and budget files.

Role	Normal Content Categories
Engineer	Engineering, IT/Security, Public/Internal General
Lawyer	Legal, Management, Public/Internal General
HR	HR, Personal Information, Public/Internal General
Finance	Finance, Management, Public/Internal General
Manager	Management, Finance, HR, Sales
IT Admin	IT/Security, Engineering, Credentials/Secrets
Sales	Sales, Customer Data, Public/Internal General
Normal Employee	Public/Internal General

Table 32 : Role Based Examples

This information was used to generate the behavior score dataset. However, the final engine does not use a fixed permission matrix. Instead, the model predicts behavior scores based on learned patterns.

3.12.2 Behavior Score Dataset

The behavior score dataset was private from a small bank in CSV format. It contained combinations of roles, actions, categories, and sensitivity levels, along with target numerical scores.

The dataset columns were:

Column	Description
role	User role
action	Requested action
category	Detected content category
sensitivity	Detected sensitivity level
profile_match_score	How normal the behavior is for the profile
behavior_risk_score	How risky the behavior is

Table 33 : Behavior Dataset

Role	Action	Category	Sensitivity	Profile Match	Behavior Risk
Engineer	read	Engineering	MEDIUM	0.95	0.05
Engineer	write/edit	Engineering	MEDIUM	0.85	0.15
Engineer	delete	Engineering	MEDIUM	0.35	0.65
Engineer	upload/share	Finance	HIGH	0.18	0.82
Normal	read	Credentials/Secrets	CRITICAL	0.05	0.95

Table 34 : behavior dataset sample

3.12.3 Encoding and Algorithm

The Behavior AI uses categorical input features. Therefore, the text categories were transformed into numerical vectors using one-hot encoding.

The features are:

- role
- action
- category
- sensitivity

The algorithm used was: Random Forest Regressor

A regressor was selected because the goal is not to classify the behavior as simply normal or abnormal. Instead, the model produces continuous numerical scores.

The model predicts:

- profile_match_score
- behavior_risk_score

This supports the project design because the AI does not directly decide ALLOW or BLOCK. It only provides numbers for the final engine equation.

3.12.4 Behavior AI Workflow

The Behavior AI workflow is:

1. The active user role is loaded from active_profile.json.
2. The requested action is detected.

3. The Sensitivity AI provides category and sensitivity level.
4. These four values are encoded using one-hot encoding.
5. The Random Forest Regressor predicts profile match and behavior risk.
6. The scores are passed to the risk scoring module.

Example:

Input:

Engineer + delete + Engineering + MEDIUM

Output:

profile_match_score = 0.35

behavior_risk_score = 0.65

This means the category may match the Engineer role, but the delete action is still risky.

3.13 Risk Scoring Equation

After the Sensitivity AI and Behavior AI produce their scores, the DLP engine calculates a final risk score. This score determines the decision level.

The final risk score is calculated using the following equation:

$$\begin{aligned}
 final_risk = & \\
 & (content_risk \times 0.40) \\
 & + \\
 & (behavior_risk \times 0.35) \\
 & + \\
 & (action_risk \times 0.25)
 \end{aligned}$$

The three risk components are:

Component	Source	Weight
content_risk	Sensitivity AI	0.40
behavior_risk	Behavior AI	0.35
action_risk	DLP Engine	0.25

Table 35 : risk components

The weights were selected to give the largest importance to content sensitivity, while still considering user behavior and the danger level of the requested action.

3.13.1 Action Risk

Action risk is assigned directly by the DLP engine. It reflects the relative danger of each file interaction type.

Action	Action Risk Score
read	0.10
download	0.30
write/edit	0.45
upload/share	0.75
delete	0.90

Table 36 : Action risk score

This means that even if the file category is normal for the user role, dangerous actions such as delete or upload/share increase the final risk score.

3.13.2 Decision Thresholds

The calculated final risk is mapped to one of four decisions:

Final Risk Range	Decision
< 0.40	ALLOW
0.40 – < 0.55	LOG
0.55 – < 0.70	WARN
≥ 0.70	BLOCK

Table 37 : Decision Score

```

Windows PowerShell
Sensitivity Vectorizer      : sensitivity_vectorizer.pkl
Sensitivity Category Model  : sensitivity_category_model.pkl
Sensitivity Level Model     : sensitivity_level_model.pkl
Sensitivity Risk Model      : sensitivity_risk_model.pkl

[06:34:49.863] HTTP(S) proxy listening at *:8888.
[06:35:49.434] [192.168.1.30:63141] client connect
[06:35:49.463] [192.168.1.30:63141] server connect 192.168.1.10:8000

=====
DLP ENGINE V4 DECISION - SENSITIVITY AI + BEHAVIOR AI + ENGINE EQUATION
=====
User      Role   ProfileMatch ContentRisk Action BehaviorRisk File ActionRisk FinalRisk Decision Category Enforcement Sensitivity Catconf Lvlconf
-----
s1        Sales  0.78        1.0        read  0.31        customer_database_extract.txt 0.1 0.53 WARN Customer Data WARN_ALLOWED CRITICAL 0.85 0.86

Sensitive / critical Findings:
credit card, credit-card-like-number, customer id, purchase history

Equation:
final_risk = (content_risk * 0.40) + (behavior_risk * 0.35) + (action_risk * 0.25)

Reason:
Sensitivity AI detected category 'Customer Data' with sensitivity 'CRITICAL', content risk 1.0, category confidence 0.85, and level confidence 0.86. Sensitivity AI prediction was adjusted by critical keyword safety override. Behavior AI score output: profile_match=0.78, behavior_risk=0.31. The DLP Engine assigned action risk 0.1 for action 'read'. The DLP Engine calculated final risk 0.53. Critical-content policy result: critical override: CRITICAL content with action 'read' must be at least WARN.. Final decision: WARN. Final enforcement: WARN_ALLOWED.

=====
192.168.1.30:63141: GET http://192.168.1.10:8000/customer_database_extract.txt?action=read HTTP/1.1
<< HTTP/1.0 200 OK 273b
[06:35:49.727] [192.168.1.30:63141] server disconnect 192.168.1.10:8000
[06:35:49.729] [192.168.1.30:63141] client disconnect
  
```

Figure 22 : Final risk in real time

Example calculation:

content_risk = 0.50

behavior_risk = 0.65

action_risk = 0.90

$$\text{final_risk} = (0.50 \times 0.40) + (0.65 \times 0.35) + (0.90 \times 0.25)$$

$$\text{final_risk} = 0.20 + 0.2275 + 0.225$$

$$\text{final_risk} = 0.6525$$

The result is approximately: 0.65 → WARN

If the action is delete, the enforcement policy may still block the request even if the decision is WARN.

This equation-based approach makes the system explainable because every final decision can be traced back to visible numerical scores.

3.14 Enforcement Policy

The DLP engine separates the risk decision from the final enforcement action. The decision is calculated using the risk equation, while the enforcement policy determines what actually happens to the request.

This separation is important because not every warning should be treated the same way. A warning for a read action may be allowed with logging, while a warning for a delete or upload/share action may be blocked.

3.14.1 Decision vs Enforcement

Term	Meaning
Decision	Risk level result from the equation and critical policy
Enforcement	Actual action applied to the HTTP response

Possible decisions:

- ALLOW
- LOG
- WARN
- BLOCK

Possible enforcement outputs:

- ALLOW
- LOG_ALLOWED
- WARN_ALLOWED
- BLOCKED_BY_WARN_POLICY
- BLOCK

3.14.2 Enforcement Rules

Decision	Action	Enforcement
ALLOW	Any action	ALLOW
LOG	Any action	LOG_ALLOWED
WARN	read/download/write-edit	WARN_ALLOWED
WARN	delete/upload-share	BLOCKED_BY_WARN_POLICY

BLOCK	Any action	BLOCK
-------	------------	-------

Table 38 : Enforcement Rules

Example:

Decision = WARN

Action = delete

Enforcement = BLOCKED_BY_WARN_POLICY

This means the engine identified the request as warning-level risk, but because the action is dangerous, the enforcement module blocks it.

3.14.3 Critical Content Policy

The engine also applies strict rules for critical data. This protects credentials, personal information, and customer records even if the risk equation produces a lower score.

Condition	Policy
Category is Credentials/Secrets	BLOCK
Sensitivity is CRITICAL and action is write/edit	BLOCK
Sensitivity is CRITICAL and action is delete	BLOCK
Sensitivity is CRITICAL and action is upload/share	BLOCK
Sensitivity is CRITICAL and action is read/download	At least WARN

Table 39 : content policy

This policy ensures that critical content cannot be silently allowed.

Example:

File	Action	Result
it_admin_credentials_note.txt	read	BLOCK
customer_database_extract.txt	upload/share	BLOCK
engineering_project_alpha.txt	read	ALLOW
engineering_project_alpha.txt	delete	BLOCKED_BY_WARN_POLICY

3.15 Chapter Summary

This chapter documented the practical implementation of the proposed AI-assisted DLP system. The system was deployed in an isolated LAN environment consisting of three machines: an internal file server, a DLP gateway, and a user endpoint. The environment was configured using static IP addresses and verified through connectivity tests.

The implementation began with the setup of a Python-based file server on PC 1 and the generation of representative company files. These files covered multiple categories such as Engineering, Legal, HR, Finance, IT/Security, Credentials, Customer Data, and Personal Information. PC 2 was then configured with MITMproxy to intercept HTTP traffic from PC 3 before forwarding it to PC 1.

The final DLP engine was designed as a modular system. It includes request processing, action detection, Sensitivity AI, Behavior AI, risk scoring, enforcement policy, and structured logging. The Sensitivity AI analyzes file content and predicts category, sensitivity level, and content risk. The Behavior AI evaluates whether the requested action matches the selected user profile and produces behavior risk scores.

Chapter Four

-Testing, Results and Evaluation-

4.1 Introduction

This chapter presents the testing and evaluation phase of the proposed AI-assisted Data Loss Prevention system. After completing the development and execution of the system components described in the previous chapter, the next step was to validate the system behavior under different user roles, file categories, sensitivity levels, and requested actions.

The main objective of this chapter is to demonstrate that the implemented DLP system can make context-aware decisions rather than applying a fixed allow or block policy to all requests. The system was tested using multiple user profiles, representative company files, and several file interaction actions. Each request was processed by the DLP engine, which combined the outputs of the Sensitivity AI, the Behavior AI, and the action risk score to calculate the final risk.

The evaluation focuses on the following aspects:

- Whether the system correctly allows normal low-risk interactions.
- Whether the system detects and blocks high-risk sensitive content.
- Whether the system reacts differently based on the selected user role.
- Whether dangerous actions such as delete and upload/share increase the risk.
- Whether the final decisions are explainable through logged scores.

The testing process was conducted inside the same isolated LAN environment described in Chapter 3. PC 3 was used to generate requests, PC 2 was used as the DLP gateway and AI analysis point, and PC 1 hosted the internal company files. The DLP engine logged all decisions into structured JSONL logs, which were later exported into CSV reports for analysis.

This chapter includes the design of the test matrix, the testing methodology, representative scenarios, AI model evaluation, decision report analysis, final execution results, and a discussion of the limitations observed during testing.

4.2 Testing Methodology

The testing methodology was designed to validate the system from both functional and analytical perspectives. Functionally, the test verifies whether the DLP gateway can intercept requests and enforce decisions correctly. Analytically, it evaluates whether the decisions are affected by the role, action, file category, and sensitivity level.

The testing flow followed the same operational path used in the final system:

PC 3 → PC 2 → PC 1

PC 3 acted as the user machine and generated file interaction requests. These requests were explicitly routed through PC 2 using the proxy option in curl.exe. PC 2 received the request through MITMproxy and passed it to the DLP engine. The engine analyzed the file content, evaluated the selected role and action, calculated the final risk, and applied the

enforcement policy. If the request was allowed, it was forwarded to PC 1. If it was blocked, PC 2 returned a DLP block message instead of the original file content.

The general command format used for testing was: ***curl.exe --proxy 192.168.1.20:8888 "http://192.168.1.10:8000/file_name.txt?action=action_name"***

For example, the following command simulates an Engineer reading an engineering file:

```
PS C:\Users\Administrator> curl.exe --proxy 192.168.1.20:8888 "http://192.168.1.10:8000/customer_database_extract.txt?action=read"
```

Figure 23 : Basic Used Command - Read

The following command simulates a delete action:

```
engineering_project_alpha.txt | action=delete
192.168.1.10:8000/engineering_project_alpha.txt?action=delete
```

Figure 24 : Basic Used Command - Delete

During testing, the active user profile was selected on PC 2 using the profile selection script. After selecting the role, the DLP engine was restarted so it could load the updated active profile from active_profile.json.

The testing cycle for each role was:

1. Select the active role using select_profile.py.
2. Start the DLP engine on PC 2.
3. Run the test script from PC 3.
4. Allow PC 2 to process and log all decisions.
5. Export the decision logs into CSV reports.
6. Analyze the results by role, action, category, sensitivity, and enforcement.

Component	Purpose
PC 1	Hosts internal company files
PC 2	Runs MITMproxy, DLP engine, and AI models
PC 3	Sends test requests through PC 2
select_profile.py	Selects the active user role
run_role_action_tests.ps1	Executes the automated role/action test matrix
decision_logs_v4.jsonl	Stores DLP decisions in structured JSONL format
CSV reports	Used for decision analysis and evaluation

Table 40 : Testing Components

The testing was performed using both manual and automated requests. Manual requests were used to confirm specific scenarios, such as allowing normal engineering access and blocking credential files. Automated testing was then used to run a larger matrix covering multiple roles, actions, and file categories.

4.3 Test Matrix Design

A test matrix was created to evaluate the DLP system across multiple combinations of user roles, file categories, and requested actions. The purpose of this matrix was to verify that the system behaves dynamically and does not apply the same decision to all requests.

The matrix was based on three dimensions:

1. User role
2. File category
3. Requested action

The eight user roles were:

1. Engineer
2. Lawyer
3. HR
4. Finance
5. Manager
6. IT Admin
7. Sales
8. Normal Employee

```
=====
DLP ACTIVE PROFILE SELECTOR
=====
Enter username, example Ahmed: Rami

Select role/profile:

1. Engineer
2. Lawyer
3. HR
4. Finance
5. Manager
6. IT Admin
7. Sales
8. Normal Employee

Enter choice number: 5

Active profile saved:
{
  "user": "Rami",
  "role": "Manager"
}

File created: active_profile.json
```

Figure 25 : Setting up the role for testing

The five supported actions were:

- Read
- Download
- write/edit
- delete
- upload/share

```

Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Administrator> curl.exe --proxy 192.168.1.20:8888 "http://192.168.1.10:8000/customer_database_extract.txt?action=read"
CUSTOMER DATABASE EXTRACT

Customer records:
Name: John Smith
Email: john.smith@example.com
Phone: 555-0101
Credit Card: 4111-1111-1111-1111
Customer ID: CUST-9001
Purchase history: enterprise subscription

Classification: CRITICAL
Department: Customer Data
PS C:\Users\Administrator> curl.exe --proxy 192.168.1.20:8888 "http://192.168.1.10:8000/customer_database_extract.txt?action=upload_share"
DLP BLOCKED

The DLP Engine blocked this action based on AI scores and risk policy.

Decision: BLOCK
Enforcement: BLOCK
Action: upload/share
Category: Customer Data
Sensitivity: CRITICAL
Final Risk: 0.77
Reason: Critical override: CRITICAL content with dangerous action 'upload/share' is blocked.
PS C:\Users\Administrator>

```

Figure 26 : Manual Test

The selected representative files covered the main categories used in the project.

Category	File Used in Test
Public/Internal General	general_company_policy.txt
Engineering	engineering_project_alpha.txt
Legal	legal_client_contract.txt
HR	hr_employee_records.txt
Finance	finance_payroll_report.txt
Sales	sales_customer_pipeline.txt
IT/Security	it_security_incident_report.txt
Credentials/Secrets	it_admin_credentials_note.txt
Customer Data	customer_database_extract.txt
Personal Information	personal_identity_records.txt

Table 41 : Representative Test Files

The resulting test matrix was: **8 roles × 10 files × 5 actions = 400 test cases**

Additional manual tests were also performed during validation, so the final exported decision logs contained approximately 480 records.

Dimension	Count	Values
Roles	8	Engineer, Lawyer, HR, Finance, Manager, IT Admin, Sales, Normal Employee
Files	10	Representative files from the main company categories
Actions	5	read, download, write/edit, delete, upload/share

Expected test cases	400	$8 \times 10 \times 5$
---------------------	-----	------------------------

Table 42 : Test Matrix Dimensions

- The matrix was executed using a PowerShell script on PC 3:
run_role_action_tests.ps1
- The script automatically generated requests using the following URL format:
http://192.168.1.10:8000/file_name.txt?action=action_name
- and sent them through PC 2 using: --proxy 192.168.1.20:8888

This allowed the DLP engine to process every role/file/action combination and produce structured logs for later analysis.

The test matrix was designed to prove the following points:

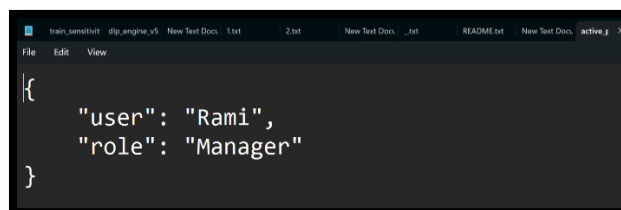
- A normal role-category combination should have lower behavior risk.
- A mismatched role-category combination should increase behavior risk.
- Dangerous actions should increase final risk.
- Critical content should be blocked or warned regardless of role.
- The same file can produce different results depending on the selected action.
- The same action can produce different results depending on the file sensitivity.

4.4 Role-Based Test Scenarios

Role-based testing was performed to verify that the DLP system responds differently depending on the selected user profile. The purpose of this test category is to confirm that the system does not only evaluate the file content, but also considers whether the requested interaction is normal for the active role.

The active role was selected on PC 2 using: **select_profile.py**

After selecting a role, the DLP engine loaded the role from: **active_profile.json**



```
{
  "user": "Rami",
  "role": "Manager"
}
```

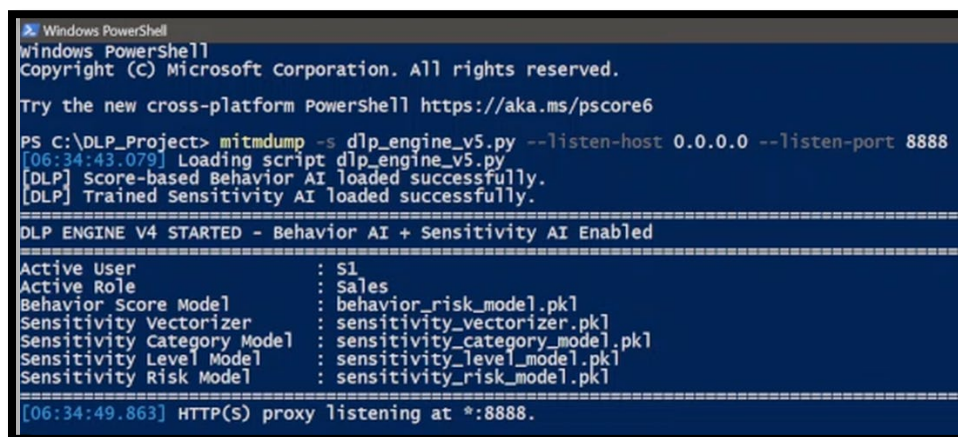
Figure 27 : Active Profile.JSON

The same files and actions were then tested from PC 3 using the automated matrix script. For every request, the Behavior AI received the following inputs:

- role
- action
- detected category
- detected sensitivity level

and produced:

- profile_match_score
- behavior_risk_score



```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\DLP_Project> mitmdump -s dlp_engine_v5.py --listen-host 0.0.0.0 --listen-port 8888
[06:34:43.079] Loading script dlp_engine_v5.py
[DLP] Score-based Behavior AI loaded successfully.
[DLP] Trained Sensitivity AI loaded successfully.

=====
DLP ENGINE V4 STARTED - Behavior AI + Sensitivity AI Enabled
=====
Active User           : S1
Active Role           : Sales
Behavior Score Model  : behavior_risk_model.pkl
Sensitivity Vectorizer : sensitivity_vectorizer.pkl
Sensitivity Category Model : sensitivity_category_model.pkl
Sensitivity Level Model : sensitivity_level_model.pkl
Sensitivity Risk Model : sensitivity_risk_model.pkl
=====
[06:34:49.863] HTTP(S) proxy listening at *:8888.
```

Figure 28 : DLP Listening to a user with role Sales

This allowed the system to evaluate whether the user action was consistent with the expected behavior of that role.

For example, when the selected role is Engineer, reading an engineering file should be considered more normal than reading finance payroll data or credential files. Similarly, when the selected role is HR, interacting with employee records is more expected than interacting with source-code or finance data.

Scenario	Expected Behavior
Engineer reads Engineering file	Low behavior risk
Engineer reads Finance payroll file	Higher behavior risk
HR reads employee records	Lower behavior risk
Normal Employee reads credentials file	Very high behavior risk
Finance reads finance report	Lower behavior risk
Sales reads customer pipeline file	Lower behavior risk
Lawyer reads legal contract	Lower behavior risk
IT Admin reads IT/Security file	Lower behavior risk

Table 43 : Role-Based Scenario Examples

This test category is important because it demonstrates the difference between a fixed permission rule and a behavior-aware DLP decision. The system does not simply check whether a role is allowed or denied. Instead, it calculates a risk score based on how well the interaction matches the learned behavior profile.

For example: Engineer + read + Engineering + MEDIUM is expected to produce a high profile match score and a low behavior risk score.

In contrast: Engineer + upload/share + Finance + HIGH is expected to produce a lower profile match score and a higher behavior risk score.

The following table shows the conceptual difference:

Role	File Category	Action	Expected Behavior Risk
Engineer	Engineering	read	Low
Engineer	Finance	read	Medium/High
Engineer	Credentials/Secrets	read	Very High
HR	HR	read	Low
HR	Engineering	write/edit	Medium/High
Finance	Finance	download	Low/Medium
Normal Employee	Public/Internal General	read	Low
Normal Employee	HR	read	High

Table 44 : Expected Behavior Risk by Role Context

The role-based testing confirmed that the behavior model contributes to the final risk calculation. Even when two files have similar sensitivity levels, the role context can change the final decision because the profile match score and behavior risk score differ.

4.5 Action-Based Test Scenarios

Action-based testing was performed to verify that the DLP system evaluates the requested operation, not only the file content or user role. In a real organizational environment, different actions on the same file do not have the same level of risk. Reading a document is usually less dangerous than deleting it or sharing it with another location. For this reason, the DLP engine assigns a specific action risk score to each supported action.

The supported actions in the experiment were:

- read
- download
- write/edit
- delete
- upload/share

During laboratory testing, these actions were simulated using URL query parameters. For example, reading an engineering file was tested using: **curl.exe --proxy 192.168.1.20:8888 "http://192.168.1.10:8000/engineering_project_alpha.txt?action=read"**

while deleting the same file was simulated using: **curl.exe --proxy 192.168.1.20:8888 "http://192.168.1.10:8000/engineering_project_alpha.txt?action=delete"**

Although both requests target the same file, the DLP engine processes them differently because the requested action changes the risk level. A read action has a low action risk, while delete and upload/share are treated as high-risk actions. This allows the system to produce different decisions for the same file depending on the operation being attempted.

For example, when the active role is Engineer and the requested file is an Engineering document, the read action is expected to be allowed because the file category matches

the role and the action is low-risk. However, if the same user attempts to delete or upload/share the same file, the final risk increases because the action itself is dangerous. In such cases, the system may produce a WARN decision or enforce a block through the warning policy.

This behavior confirms that the system is action-aware. It does not simply classify a file as allowed or blocked permanently. Instead, each interaction is evaluated independently using the file content, the user role, and the requested action.

A key result of this test was the enforcement of dangerous actions. If the engine calculates a WARN decision for a delete or upload/share action, the final enforcement becomes:

BLOCKED_BY_WARN_POLICY

This means that the request was not fully classified as BLOCK by the risk equation, but the enforcement policy still prevented it because the action is considered dangerous.

Example: Decision = WARN , Action = delete ,Enforcement = BLOCKED_BY_WARN_POLICY

This test demonstrates that the final system supports action-specific control, which is an essential requirement for a practical DLP system.

4.6 Critical Content Test Scenarios

Critical content testing was performed to verify that the system applies stricter protection to highly sensitive data. This part of the evaluation focused on files containing credentials, customer records, and personal information. These data types are considered critical because their leakage can directly lead to account compromise, privacy violations, or serious organizational damage.

The main critical files used in the test were:

- it_admin_credentials_note.txt
- customer_database_extract.txt
- personal_identity_records.txt

The credentials file contains patterns such as passwords, tokens, API keys, and administrative account references. The customer data file contains customer identifiers, contact information, purchase history, and payment-related data. The personal information file contains identity-related fields such as SSN, passport number, national ID, and home address.

The DLP engine applies an additional critical-content policy after calculating the normal risk score. This policy ensures that critical content is not silently allowed, even if the behavior score or action score alone does not push the final risk into the BLOCK range.

For example, the following command tests reading a credentials file:

```
curl.exe --proxy 192.168.1.20:8888
```

```
"http://192.168.1.10:8000/it_admin_credentials_note.txt?action=read"
```

The expected result is BLOCK, because credentials and secrets are treated as a strict blocking category in the final system.

Another example is uploading or sharing customer data:

```
curl.exe --proxy 192.168.1.20:8888
```

```
"http://192.168.1.10:8000/customer_database_extract.txt?action=upload_share"
```

The expected result is also BLOCK, because the content is critical and the requested action is dangerous.

The critical-content policy follows these principles:

- Credentials/Secrets are blocked.
- CRITICAL content with upload/share, delete, or write/edit is blocked.
- CRITICAL content with read or download must be at least warned.

This policy is applied by the DLP engine, not directly by the AI model. The Sensitivity AI identifies the content category and sensitivity level, while the DLP engine applies the final enforcement rule.

The critical content test confirmed that the system protects highly sensitive files more strictly than normal business documents. It also proved that critical data can override ordinary scoring behavior. For example, even if a user role is normally related to IT/Security, a file containing passwords or tokens is still blocked under the Credentials/Secrets policy.

4.7 AI Model Evaluation

The proposed DLP system depends on two AI components: the **Behavior AI model** and the **Sensitivity AI model**. The purpose of this section is to evaluate the role of each model and explain how its output contributes to the final DLP decision.

A key design principle in this project is that the AI models do **not** directly make enforcement decisions. They do not independently decide whether a request should be allowed or blocked. Instead, each model produces numerical outputs or classification results that are passed to the DLP engine. The DLP engine then combines these values using the final risk equation and applies the enforcement policy.

This separation improves explainability because every final decision can be traced back to clear intermediate values:

- content_risk_score
- behavior_risk_score
- action_risk_score
- final_risk_score
- decision

- enforcement

4.7.1 Behavior AI Evaluation

The Behavior AI model was trained to evaluate whether a requested action is normal for a selected user role. It receives structured information about the user role, requested action, detected content category, and sensitivity level.

The model input is: role , action , category ,sensitivity_level

The model output is: profile_match_score , behavior_risk_score

The model was trained using a generated behavior score dataset. This dataset represented expected behavior patterns for the roles used in the experiment, such as Engineer, HR, Finance, Lawyer, IT Admin, Sales, Manager, and Normal Employee.

The input features were categorical, so one-hot encoding was used to convert them into numerical form. After encoding, a **Random Forest Regressor** was trained to predict the two numerical outputs.

The use of a regressor was intentional. A classifier that outputs only labels such as normal, unusual, or abnormal is less flexible for risk calculation. A regression model provides continuous scores, which are more suitable for a risk-based DLP engine.

For example, the model can represent the difference between a slightly unusual action and a highly abnormal action:

This output is then passed to the DLP engine, where it is combined with content risk and action risk.

The main advantage of this model is that it allows the system to consider the user context. The same file may produce different behavior scores depending on the selected role. This is important because a finance report may be normal for a Finance profile, but unusual for an Engineer or Normal Employee profile.

4.7.2 Sensitivity AI Evaluation

The Sensitivity AI model was trained to analyze the actual text content of a file. Its purpose is to classify the content category, predict the sensitivity level, and estimate the numerical content risk score.

The model input is: file_text

The model output is: category ,sensitivity_level ,content_risk_score

The training dataset contained text samples representing different business categories, such as Engineering, Legal, HR, Finance, Sales, IT/Security, Customer Data, Credentials/Secrets, and Personal Information.

Before training, the file text was transformed into numerical features using **TF-IDF vectorization**. TF-IDF was selected because it gives higher importance to meaningful terms that distinguish one category from another. For example, terms such as password, token, and api_key are important indicators for Credentials/Secrets, while terms such as payroll, salary, and tax are important for Finance.

The Sensitivity AI used three trained models:

- Random Forest Classifier → category prediction
- Random Forest Classifier → sensitivity level prediction
- Random Forest Regressor → content risk score prediction

This structure allowed the system to produce both classification outputs and a numeric score. The category and sensitivity level help describe the content, while the content risk score is used directly in the final DLP equation.

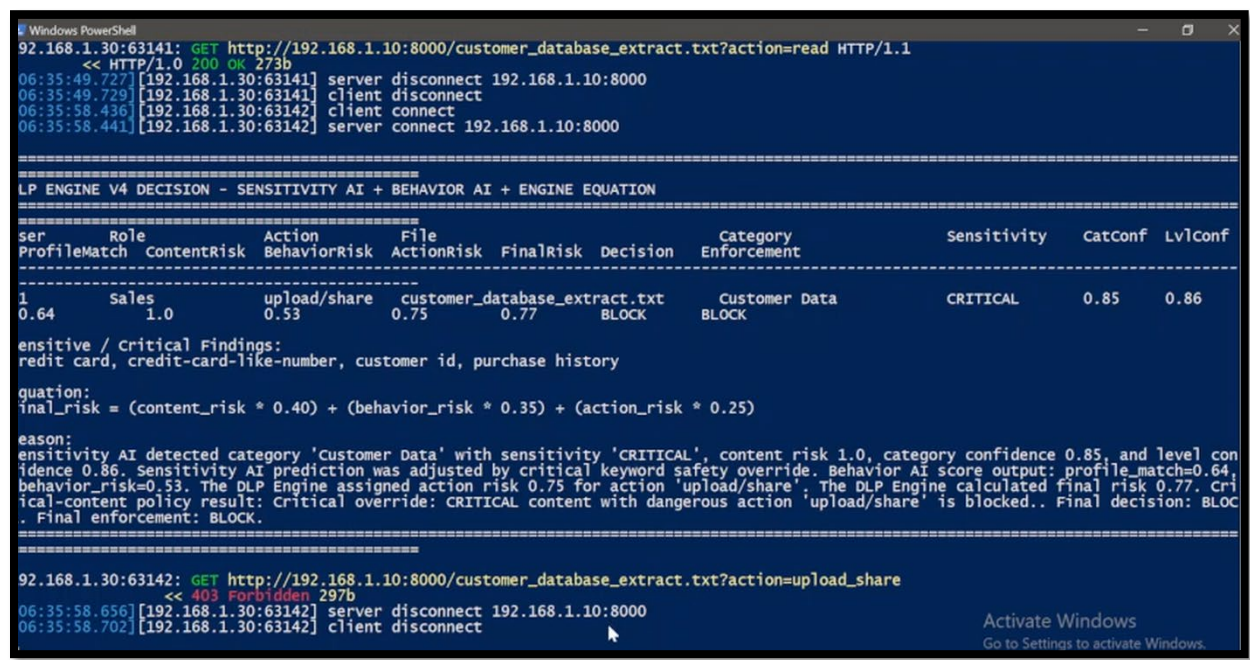


Figure 29 : Action Rejection / Blocking based on risk score

4.7.3 AI Output Role in the Final Decision

Both AI components contribute to the final DLP decision, but neither model directly blocks a request.

The Sensitivity AI answers: What type of content is this, and how sensitive is it?

The Behavior AI answers: Is this action normal for this user role?

The DLP engine answers: Based on content risk, behavior risk, and action risk, what should be done?

This structure keeps the final decision explainable and policy-controlled.

4.8 DLP Decision Report Analysis

After executing the role/action test matrix, the DLP engine generated structured decision logs. These logs were exported into CSV reports to simplify analysis and documentation.

The main log file was stored in JSON Lines format. Each line represented one processed request and included the user role, requested action, file name, detected category, sensitivity level, risk scores, decision, enforcement result, and explanation.

The exported CSV reports included:

- decision_report.csv
- decision_summary_by_role.csv
- decision_summary_by_action.csv
- decision_summary_by_category.csv
- decision_summary_by_sensitivity.csv
- decision_summary_by_enforcement.csv
- decision_summary_role_decision.csv
- decision_summary_role_action.csv

The main report, decision_report.csv, provides the most detailed view of the system behavior. It contains one row per request and includes all values used in the final decision.

The most important columns are:

- role
- action
- file
- category
- sensitivity_level
- profile_match_score
- content_risk_score
- behavior_risk_score
- action_risk_score
- final_risk_score
- decision
- enforcement
- reason

These fields make the system auditable. For example, when a request is blocked, the report shows whether the block was caused by high content risk, abnormal behavior, dangerous action, or a critical-content override.

The summary reports provide a higher-level view of the results. For example, the role summary shows how many requests were processed for each user role. The action

summary shows how decisions were distributed across read, download, write/edit, delete, and upload/share actions. The enforcement summary shows how many requests were allowed, logged, warned, blocked, or blocked by warning policy.

During the evaluation, the exported report contained approximately 480 records. This included the full automated test matrix and additional manual validation tests. The results included a mixture of decisions: ALLOW , LOG , WARN or BLOCK

This mixture is important because it shows that the DLP engine was not configured to blindly block all requests or blindly allow all requests. Instead, it evaluated each request based on context.

The enforcement results also included different outcomes:

- ALLOW
- LOG_ALLOWED
- WARN_ALLOWED
- BLOCKED_BY_WARN_POLICY
- BLOCK

This confirms that the enforcement layer was working separately from the decision layer. For example, a request may receive a WARN decision, but if the action is delete or upload/share, the enforcement policy may block it as BLOCKED_BY_WARN_POLICY.

The exported reports are useful for several reasons:

- They provide evidence that the system processed the test matrix.
- They show the final decision for each role/action/file combination.
- They make the risk calculation explainable.
- They can be used for screenshots in the report.
- They can support future analysis and improvement of the models.

4.9 Final Execution Results

The final execution results demonstrate that the system successfully performs role-aware, action-aware, and content-aware DLP enforcement.

Several representative scenarios were selected from the test results to show the behavior of the final system.

In the first scenario, an Engineer attempted to read an Engineering file. This request represents a normal role-category relationship and a low-risk action. The expected result is ALLOW, and the system allowed the request.

Role: Engineer

File: engineering_project_alpha.txt

Action: read

Result: ALLOW

This confirms that the system does not block normal business activity.

In the second scenario, the same Engineer attempted to delete the same Engineering file. Although the file category matches the Engineer role, the requested action is dangerous. The risk equation increases the final risk because delete has a high action risk score. If the decision reaches WARN, the enforcement policy blocks it as a dangerous warned action.

Role: Engineer

File: engineering_project_alpha.txt

Action: delete

Result: BLOCKED_BY_WARN_POLICY or BLOCK

This confirms that the same file can be handled differently depending on the requested action.

In the third scenario, an Engineer attempted to read a credentials file. The system detected the content as Credentials/Secrets with CRITICAL sensitivity. According to the critical-content policy, credentials are blocked regardless of the user role.

Role: Engineer

File: it_admin_credentials_note.txt

Action: read

Result: BLOCK

This confirms that critical secrets receive strict protection.

In the fourth scenario, customer data was tested with an upload/share action. Since customer data is critical and upload/share is a high-risk action, the request was blocked.

File: customer_database_extract.txt

Action: upload/share

Result: BLOCK

This confirms that the system protects sensitive customer information against risky movement.

In the fifth scenario, a general company policy file was accessed using a read action. The content is low sensitivity and the action is low risk. The system allowed the request.

File: general_company_policy.txt

Action: read

Result: ALLOW

This confirms that low-risk internal content remains accessible.

Together, these results show that the final system meets the main project objective. It evaluates each request using multiple factors rather than relying on one condition only.

The final system considers:

- who is performing the action
- what action is requested
- what file content is involved
- how sensitive the content is
- how normal the action is for the selected role
- how dangerous the requested action is

The final DLP engine therefore demonstrates contextual decision-making. It allows normal file access, logs or warns about moderate risk, and blocks high-risk or critical interactions.

```
192.168.1.30:63159: GET http://192.168.1.10:8000/engineering_project_alpha.txt?action=delete
<< 403 Forbidden 214b
[06:36:17.212] [192.168.1.30:63159] server disconnect 192.168.1.10:8000
[06:36:17.259] [192.168.1.30:63159] client disconnect
[06:36:17.600] [192.168.1.30:63160] client connect
[06:36:17.617] [192.168.1.30:63160] server connect 192.168.1.10:8000

=====
DLP ENGINE V4 DECISION - SENSITIVITY AI + BEHAVIOR AI + ENGINE EQUATION
=====
User      Role      Action      File      Category      Sensitivity      CatConf      LvlConf
ProfileMatch ContentRisk BehaviorRisk ActionRisk FinalRisk Decision Enforcement
-----
S1        Sales    upload/share engineering_project_alpha.txt Engineering MEDIUM      0.8      0.86
0.45      0.51      0.58      0.75      0.59      WARN      BLOCKED_BY_WARN_POLICY

Sensitive / Critical Findings:
None

Equation:
Final_risk = (content_risk * 0.40) + (behavior_risk * 0.35) + (action_risk * 0.25)

Reason:
Sensitivity AI detected category 'Engineering' with sensitivity 'MEDIUM', content risk 0.51, category confidence 0.8, and level confidence 0.86. Sensitivity AI prediction used without critical override. Behavior AI score output: profile_match=0.45, behavior_risk=0.58. The DLP Engine assigned action risk 0.75 for action 'upload/share'. The DLP Engine calculated final risk 0.59. Critical-content policy result: No override. Final decision: WARN. Final enforcement: BLOCKED_BY_WARN_POLICY.

=====
192.168.1.30:63160: GET http://192.168.1.10:8000/engineering_project_alpha.txt?action=upload_share
<< 403 Forbidden 306b
[06:36:17.948] [192.168.1.30:63160] server disconnect 192.168.1.10:8000
[06:36:17.999] [192.168.1.30:63160] client disconnect
```

Figure 30 : DLP Warning

Chapter Five

-Conclusion and Future Work-

5.1 Chapter Introduction

This chapter presents the final conclusion of the project and summarizes the work completed throughout the development of the proposed AI-assisted Data Loss Prevention system. The project aimed to design and implement a practical DLP solution for a local area network environment, with a focus on content sensitivity, user-role behavior, and action-based risk evaluation.

The previous chapters introduced the theoretical background of DLP systems, described the development and execution process, and evaluated the final implementation using a controlled testing matrix. This chapter summarizes the achieved objectives, highlights the main contributions of the project, discusses the challenges faced during development, explains the current limitations of the system, and proposes future improvements.

The completed system demonstrates that it is possible to build a local AI-assisted DLP prototype that intercepts user file interactions, analyzes file content, evaluates role-based behavior, calculates a final risk score, and applies policy-based enforcement decisions such as ALLOW, LOG, WARN, and BLOCK.

5.2 Project Summary

The main goal of this project was to develop a web-based DLP system for an isolated LAN environment. The system was designed to monitor file interactions between a user machine and an internal server through a dedicated DLP gateway. The final implementation used three main machines:

- PC 1: Internal file server and domain controller
- PC 2: DLP gateway, MITMproxy, and AI model execution point
- PC 3: User endpoint machine

The traffic path was designed as: PC 3 → PC 2 → PC 1

This structure allowed PC 2 to inspect requests before they reached the internal server or before the response was delivered to the user. MITMproxy was used as the interception layer, while the DLP engine performed request processing, content analysis, behavior scoring, risk calculation, enforcement, and logging.

The system was developed around two AI components:

1. Sensitivity AI
2. Behavior AI

The Sensitivity AI analyzes file content and predicts:

- Category
- sensitivity_level
- content_risk_score

The Behavior AI analyzes the relationship between the user role, requested action, detected category, and sensitivity level. It predicts:

- profile_match_score
- behavior_risk_score

The final decision is made by the DLP engine using a weighted risk equation:

$$\begin{aligned} final_risk = & \\ & (content_risk \times 0.40) \\ & + \\ & (behavior_risk \times 0.35) \\ & + \\ & (action_risk \times 0.25) \end{aligned}$$

The result is mapped to one of four decisions: ALLOW , LOG, WARN or BLOCK

The enforcement policy then determines the actual action applied to the response. For example, a WARN decision for a delete or upload/share action may still be blocked because those actions are considered dangerous.

5.3 Achieved Objectives

The project achieved its main technical objectives. The final system was able to intercept user requests, analyze content, evaluate user-role behavior, calculate risk, enforce decisions, and generate structured reports.

5.3.1 Internal LAN DLP Gateway

A working DLP gateway was implemented on PC 2 using MITMproxy. The gateway successfully intercepted requests from PC 3 before forwarding them to PC 1. This confirmed that the DLP system could operate as an inspection point inside a local area network.

The gateway was able to:

- Receive user requests through a proxy.
- Inspect request parameters.
- Access the server response.
- Modify the response when blocking was required.
- Log every processed interaction.

This objective was important because the entire system depends on PC 2 acting as the control point between the user and the file server.

5.3.2 Company File Server

A lightweight internal file server was deployed on PC 1. The file server hosted generated company documents representing multiple departments and sensitivity levels. These files included Engineering, Finance, HR, Legal, Sales, IT/Security, Credentials/Secrets, Customer Data, and Personal Information.

The server allowed controlled testing of file interactions and provided realistic content for the Sensitivity AI model.

5.3.3 Content Sensitivity Analysis

The system successfully included a trained Sensitivity AI model. This model analyzed file text and produced a category, sensitivity level, and content risk score.

The sensitivity component allowed the system to detect files such as:

- Engineering documents
- Finance reports
- HR employee records
- Legal contracts
- Credentials and secrets
- Customer data
- Personal information

This achieved one of the main project requirements: detecting sensitive content based on the actual file content rather than file extension.

5.3.4 Role-Based Behavior Analysis

The Behavior AI model was trained to evaluate whether a requested file interaction is normal for a selected user role. The model produced numerical scores instead of directly deciding whether to allow or block the action.

This approach allowed the system to evaluate cases such as:

- Engineer reading Engineering files
- Engineer accessing Finance files
- HR reading employee records
- Normal Employee accessing Credentials/Secrets
- Sales accessing Customer Data

The use of behavior scoring improved the contextual awareness of the system.

5.3.5 Action-Based Risk Evaluation

The DLP engine considered the requested action as an important risk factor. The supported actions were: Read , download , write/edit , delete , upload/share

Each action was assigned a risk score. Read was considered low risk, while delete and upload/share were considered high-risk actions. This allowed the system to treat the same file differently depending on the requested operation.

For example:

Engineer + Engineering file + read → ALLOW

Engineer + Engineering file + delete → BLOCKED_BY_WARN_POLICY or BLOCK

This fulfilled the requirement that the system should evaluate each action separately.

5.3.6 Explainable Decision Engine

The final decision was not made directly by the AI models. Instead, the AI models generated scores and the DLP engine applied a transparent equation. This makes the decision process explainable and easier to justify.

Each decision included:

- content_risk_score
- behavior_risk_score
- action_risk_score
- final_risk_score
- decision
- enforcement
- reason

This supports auditing and helps administrators understand why a request was allowed or blocked.

5.3.7 Structured Logging and Reporting

The system generated structured logs in JSONL format and exported them into CSV reports. The reports included detailed decision records and summary files grouped by role, action, category, sensitivity, and enforcement.

The exported reports can be used for:

- Auditing
- Evaluation
- Demonstration
- Future model improvement
- Administrative review

This achieved the reporting objective of the project.

5.4 Main Contributions

The project provides several important contributions from a technical and practical perspective.

First, it presents a working AI-assisted DLP prototype that runs entirely inside an isolated LAN environment. The system does not depend on cloud APIs or external services, which makes it suitable for controlled laboratory testing and secure environments.

Second, the project separates AI scoring from enforcement. This is an important architectural decision. Instead of allowing an AI model to directly block a request, the models provide risk indicators, and the DLP engine makes the final decision using a clear equation and policy rules.

Third, the system combines content sensitivity and role-based behavior. A request is not evaluated only by the file content or only by the user role. Instead, the decision considers:

- what the content is
- how sensitive it is
- who is requesting it
- what action is requested
- how risky the behavior is

Fourth, the project includes action-specific enforcement. The same user and file can produce different results depending on whether the user is reading, downloading, editing, deleting, or sharing the file.

Fifth, the system produces explainable logs and reports. Every decision is recorded with its risk scores and reason, which makes the system easier to evaluate and audit.

5.5 Challenges Faced

Several technical challenges were encountered during the development and execution of the system.

5.5.1 Proxy Routing and Traffic Path Verification

One of the first challenges was ensuring that the traffic from PC 3 actually passed through PC 2 before reaching PC 1. This required verifying connectivity, configuring MITMproxy correctly, and testing proxy-based curl requests. The traffic path was validated only after confirming output on all three machines.

5.5.2 File Server Port and Permissions

The initial attempt to run the server on port 80 caused permission issues on Windows. To solve this, the internal file server was moved to port 8000. This avoided conflicts with privileged ports and allowed the Python server to run reliably.

5.5.3 Query Parameter Handling

The test environment used query parameters to simulate actions such as delete and upload/share. The server initially treated the query string as part of the file name, which caused file not found errors. This was fixed by parsing the URL and separating the actual file path from the query string.

5.5.4 PowerShell URL Construction

During automated testing, the PowerShell script initially generated incorrect URLs. This caused requests to be sent to incorrect paths. The issue was solved by using safer string formatting to build URLs correctly.

5.5.5 Designing Synthetic Datasets

Since the laboratory environment did not include real enterprise user activity or real company datasets, synthetic datasets were generated for both Behavior AI and Sensitivity AI. Designing these datasets required careful consideration to ensure that the model learned reasonable patterns without becoming unrealistic.

5.5.6 Balancing WARN and BLOCK Decisions

Another challenge was tuning the decision logic. Some interactions produced WARN instead of BLOCK, even when they involved dangerous actions. This led to the introduction of the enforcement policy layer, where WARN decisions for delete and upload/share actions can be blocked.

5.5.7 Critical Content Handling

Credentials, customer data, and personal information require stricter enforcement. The system was enhanced with a critical-content policy and a critical keyword safety override to ensure that highly sensitive content is not silently allowed.

5.6 Limitations

Although the final system successfully achieved the main objectives of the project, it still has several limitations.

5.6.1 Synthetic Datasets

The AI models were trained on synthetic datasets generated for the laboratory environment. While these datasets were useful for proof-of-concept testing, they do not fully represent real enterprise behavior or real document diversity. In a production environment, the models would need to be trained and validated using real organizational data.

5.6.2 HTTP-Focused Implementation

The current implementation focuses on HTTP traffic through MITMproxy. It does not yet inspect other common internal file transfer protocols such as SMB, FTP, or SFTP. Real

enterprise environments usually use multiple protocols, so future work should expand protocol support.

5.6.3 Simulated File Actions

Actions such as delete, upload/share, and write/edit were simulated using URL query parameters. This was suitable for controlled testing, but real enforcement would require integration with the actual file management application or protocol.

For example, a real delete operation would require handling an HTTP DELETE method or integration with a file server API.

5.6.4 Small-Scale Laboratory Environment

The system was tested in a three-machine LAN. This is sufficient for proof-of-concept validation, but it does not represent the scale, complexity, or performance requirements of a large enterprise network.

5.6.5 No Production GUI Dashboard

The current system relies mainly on console output and exported CSV reports. A production-ready DLP solution would benefit from a graphical dashboard showing alerts, user behavior, risk trends, and real-time decisions.

5.6.6 Limited User Identity Integration

The active profile was selected manually using a profile selection script. In a production system, user identity should be automatically extracted from authentication systems such as Active Directory, Kerberos, or application login sessions.

5.6.7 No Real-Time Administrator Alerting

The system logs decisions and exports reports, but it does not yet send real-time notifications to administrators through email, SIEM, or a security dashboard.

5.7 Future Work

Several improvements can be implemented in future versions of the system.

5.7.1 Support Real File Operations

Future work should replace simulated actions with real file operations. This includes supporting actual read, download, write/edit, delete, and upload/share behavior through real HTTP methods or integration with a file management system.

5.7.2 Add SMB and FTP Support

Since many organizations use SMB or FTP for internal file transfer, future versions should support these protocols. This would make the system more realistic for enterprise LAN environments.

5.7.3 Integrate with Active Directory

The system should automatically identify users and groups from Active Directory. Instead of manually selecting the active profile, the DLP engine could map authenticated users to roles and departments.

5.7.4 Build a Graphical Dashboard

A GUI dashboard would improve usability. The dashboard could display:

- live requests
- blocked actions
- risk scores
- user activity
- top sensitive categories
- enforcement summaries
- historical reports

5.7.5 Improve Sensitivity AI with Real Data

The Sensitivity AI model should be trained on larger and more realistic datasets. This would improve classification accuracy and reduce false positives and false negatives.

5.7.6 Improve Behavior AI with Real User Logs

The Behavior AI model should be improved using real user behavior logs. This would allow the system to learn actual normal behavior patterns rather than relying only on synthetic baselines.

5.7.7 Add SIEM Integration

The DLP system could be integrated with a SIEM platform. This would allow DLP events to be correlated with other security events such as failed logins, malware alerts, or suspicious network traffic.

5.7.8 Add Real-Time Alerting

Future versions should support sending real-time alerts to administrators. Possible alert channels include:

- email
- dashboard notifications
- SIEM events
- webhooks
- log monitoring systems

5.7.9 Add Policy Management Interface

Instead of hardcoding decision thresholds and enforcement rules, future work could include a policy editor. This would allow administrators to define risk thresholds, role policies, and critical-content rules through a user interface.

5.7.10 Performance and Scalability Testing

Future testing should evaluate system performance under higher traffic loads. This includes measuring latency, processing time, CPU usage, memory usage, and throughput.

5.8 Final Conclusion

This project successfully designed and implemented an AI-assisted Data Loss Prevention system for an isolated local area network environment. The system intercepts user file requests, analyzes file content, evaluates user behavior, calculates risk, applies enforcement policies, and records all decisions in structured logs.

The final system demonstrates that DLP enforcement can be improved by combining content sensitivity analysis with role-based behavior analysis. The Sensitivity AI identifies the category and sensitivity of the file content, while the Behavior AI evaluates whether the requested action is normal for the selected user role. The DLP engine then combines these outputs with action risk to calculate a final decision.

One of the most important design choices was separating AI scoring from enforcement. The AI models provide numerical indicators, but the DLP engine remains responsible for the final decision. This makes the system more explainable, auditable, and suitable for academic evaluation.

The laboratory results showed that the system can allow normal interactions, log moderate-risk events, warn about suspicious actions, and block critical or dangerous interactions. Examples include allowing an Engineer to read an Engineering file, blocking access to Credentials/Secrets, and preventing upload/share actions for critical customer data.

Although the current implementation is a proof of concept, it provides a strong foundation for future work. With support for additional protocols, real user identity integration, real enterprise datasets, a graphical dashboard, and SIEM integration, the system could be extended toward a more complete enterprise DLP solution.

Overall, the project achieved its main goal: developing a working, explainable, AI-assisted DLP prototype capable of protecting sensitive data inside a local area network using content analysis, behavior scoring, and policy-based enforcement.

References

References for chapter 1

- NIST Computer Security Resource Center. “Data Loss Prevention.” NIST defines DLP as capabilities used to identify, monitor, and protect data in use, data in motion, and data at rest through content inspection and contextual analysis.
- IBM. “What Is Data Loss Prevention?” IBM describes DLP as a security approach that helps organizations stop data leaks by tracking data and enforcing security policies.
- Gartner. “Market Guide for Data Loss Prevention.” Gartner describes DLP as a mature market and highlights the shift toward adaptive, user-centric, risk-based data security techniques.
- Palo Alto Networks. “What Is Data Loss Prevention?” This source explains DLP as monitoring and managing data at rest, in use, and in transit to prevent unauthorized access, misuse, or loss.
- NIST Special Publication 800-122. *Guide to Protecting the Confidentiality of Personally Identifiable Information (PII)*. This is useful for explaining the importance of protecting personal data and sensitive identifiers.

References for chapter 2

- NIST Computer Security Resource Center. “Data Loss Prevention.” Useful for the formal DLP definition and the idea of protecting data in use, in motion, and at rest.
- Gartner. “Market Guide for Data Loss Prevention.” Useful for state-of-the-art discussion and the shift from traditional DLP to user-centric, adaptive, risk-based approaches.
- Gartner Peer Insights. “Data Loss Prevention Reviews.” This page gives Gartner’s DLP definition and explains that DLP controls are commonly applied to data at rest and data in motion.
- Alneyadi, S., Sithirasenan, E., and Muthukkumarasamy, V. “A Survey on Data Leakage Prevention Systems.” *Journal of Network and Computer Applications*, 2016. This survey is useful for related work, DLP mechanisms, research directions, and limitations of existing DLPS methods.
- Herrera Montano, I. et al. “Survey of Techniques on Data Leakage Protection and Prevention.” *Cluster Computing*, 2022. This source is useful for data leakage protection techniques and insider-threat-oriented data leakage prevention.
- SANS Institute. “Data Loss Prevention.” This older but practical SANS reading-room paper is useful for implementation concepts, sensitive data marking, and common data exit points such as email, webmail, and FTP.

- CIS Controls v8.1, Control 3.13. “Deploy a Data Loss Prevention Solution.” This control supports the idea of using automated tools to identify sensitive data stored, processed, or transmitted through enterprise assets.
- Proofpoint. “2025 Gartner Market Guide for Data Loss Prevention.” Useful as an industry-facing source for modern DLP trends such as cloud-focused DLP, insider risk management, DSPM, and endpoint/email/security-service-edge approaches.
- Palo Alto Networks. “What Is Data Loss Prevention?” Useful for explaining DLP categories, monitoring, sensitive data protection, and policy enforcement concepts.
- IBM. “What Is Data Loss Prevention?” Useful for explaining how DLP supports policy-based access to enterprise data and prevents unauthorized leakage

References for chapter 3

- MITMproxy Documentation. “Introduction.” This source explains that MITMproxy is an intercepting proxy for HTTP/1, HTTP/2, and WebSockets, and that mitmdump is the command-line version of MITMproxy.
- MITMproxy Documentation. “Anatomy of an Addon.” This supports the use of Python addon scripts for MITMproxy-based request/response processing.
- MITMproxy Documentation. “Addon Examples.” Useful for supporting the concept that MITMproxy behavior can be extended using Python scripts/addons.
- Python Documentation. urllib.parse — Parse URLs into components. This supports the use of urlparse() to separate the file path from query parameters such as ?action=read.
- Python Documentation. http.server module. Use this for the internal file-server implementation. If you want a simple official reference, cite the Python standard library documentation for HTTP server functionality. The Python URL parsing reference above supports the query-handling part.
- scikit-learn Documentation. TfidfVectorizer. This supports the Sensitivity AI feature extraction step, where raw document text is converted into TF-IDF feature vectors.
- scikit-learn Documentation. RandomForestClassifier. This supports the category and sensitivity-level classification models used by the Sensitivity AI.
- scikit-learn Documentation. RandomForestRegressor. This supports the behavior score model and content-risk regression model.
- scikit-learn Documentation. OneHotEncoder. This supports the Behavior AI encoding process for categorical inputs such as role, action, category, and sensitivity level.
- scikit-learn Documentation. “Model Persistence.” This supports saving and loading trained model files such as .pkl files for later use by the DLP engine.
- scikit-learn Documentation. “Getting Started.” This can be used as a general reference for scikit-learn as an open-source machine-learning library supporting model fitting and supervised learning